

Codificação de Cores



Carlos Alexandre Barros de Mello
CIn/UFPE



Codificação de Cores

4.331 cores

Imagem
Original



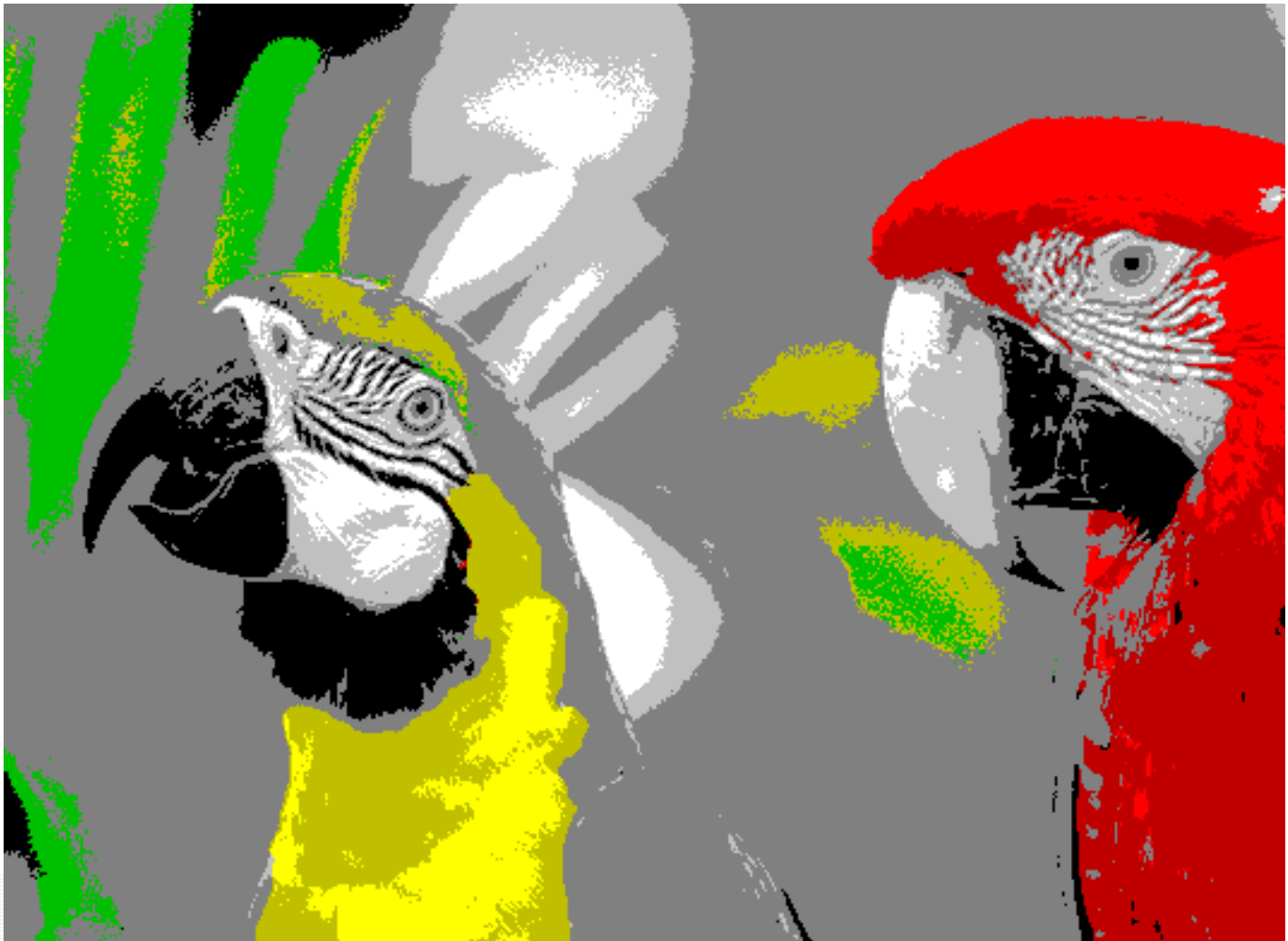
256 Cores - Recorte



256 Cores - Dithering



16 Cores - Recorte



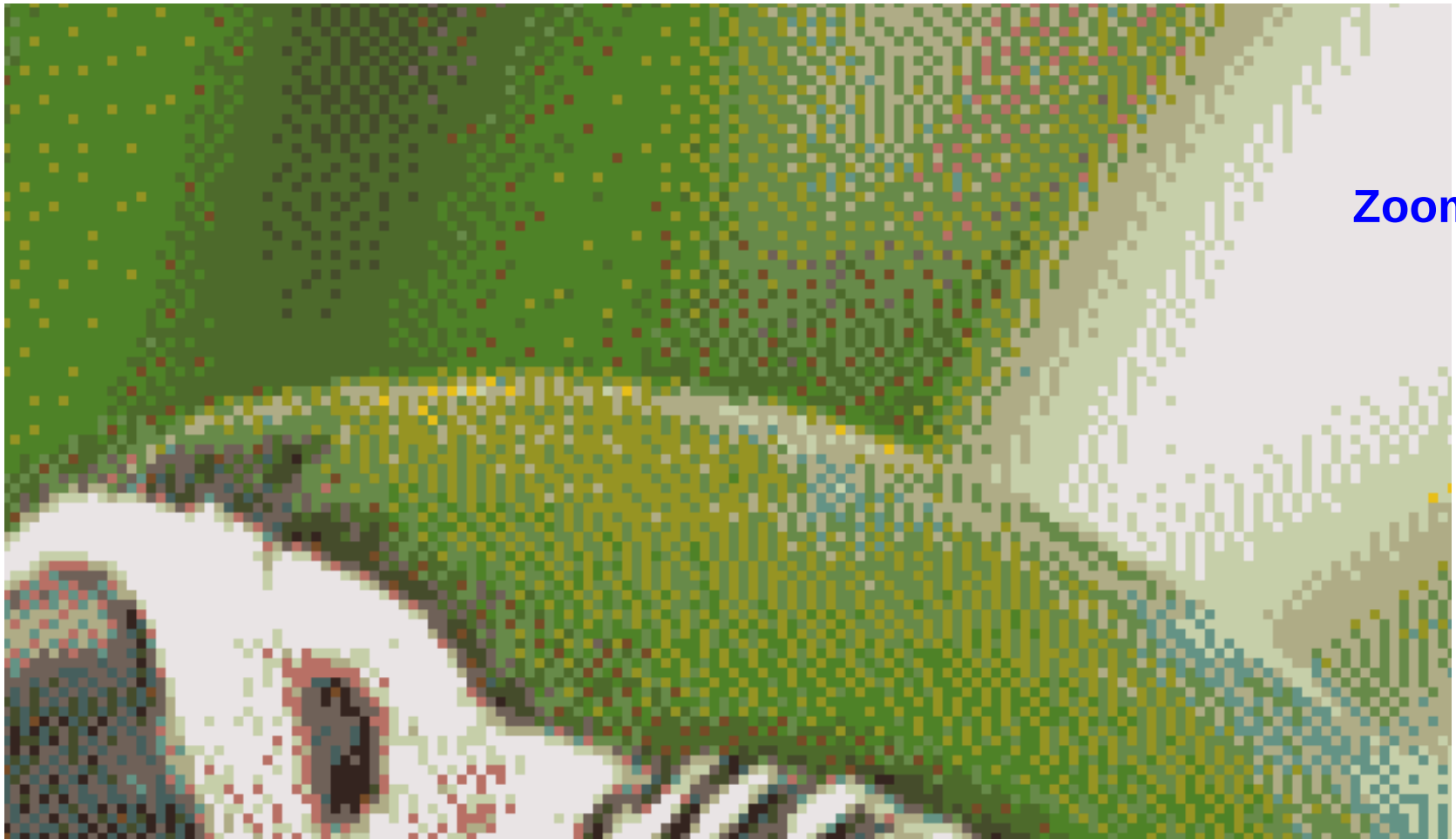
16 Cores - Dithering



16 Cores - Dithering



16 Cores - Dithering



Zoom

2 Cores - Recorte



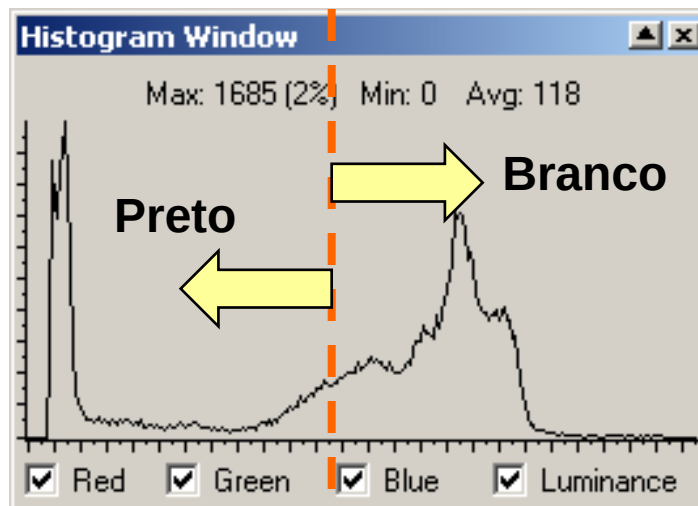
2 Cores - Recorte

- Algoritmo de Recorte

Se $\text{cor}(i) \leq 127$

Então $\text{cor}(i) = \text{Preto (0)}$

Senão $\text{cor}(i) = \text{Branco (255)}$



**Valor de Corte = 127
(Threshold, Limiar)**

2 Cores - Dithering Ordenado



2 Cores - Dithering Floyd-Steinberg



2 Cores - Dithering Floyd-Steinberg



Zoom

2 Cores - Dithering Burkes



2 Cores - Dithering Stucki



Mudança de Número de Cores

True Color -> 256 Cores



Imagem Original
(*True Color*):
35.416 cores



Imagem com 256 cores:
redução com **paleta**
Padrão, método de *cor*
mais próxima



Imagem com 256 cores:
redução com **paleta**
Padrão, método de
Dithering

Mudança de Número de Cores

True Color -> 2 Cores



Imagem Original
(True Color):
35.416 cores

Imagem com 2 cores:
redução com **paleta**
padrão método de *cor*
mais próxima



Imagem com 2 cores:
redução com **paleta**
padrão método de
Dithering

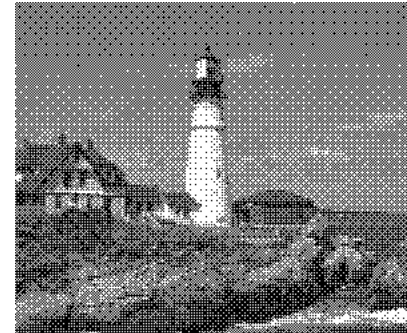


Imagem com 2 cores:
redução com **paleta**
padrão método de
Dithering



Mudança de Número de Cores

True Color -> Tons de Cinza



Imagem Original
(*True Color*)



Imagem em 256
Tons de Cinza

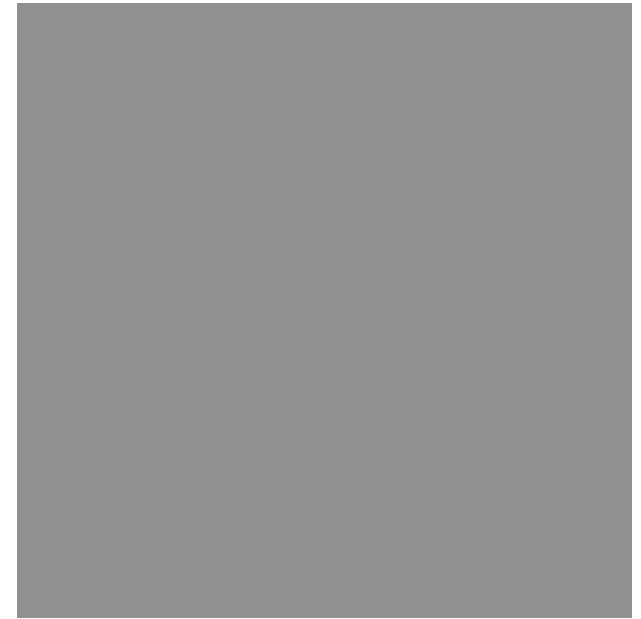
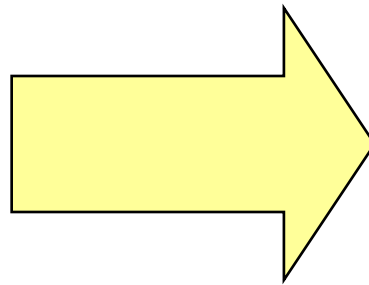
Conversão: $C = \lceil 0,29 * R + 0,59 * G + 0,11 * B \rceil$

Mudança de Número de Cores

True Color -> Tons de Cinza



(213, 115, 115)



(142, 142, 142)

$$C = \lceil 0,29*213 + 0,59*115 + 0,11*115 \rceil = 142,27$$

Dithering - Pontilhamento

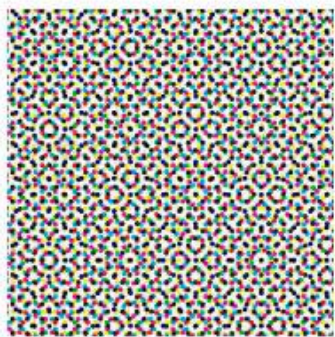
- Processo de *quantização* de cor
- Grande relevância na exibição de uma imagem digital em diferentes tipos de dispositivos gráficos
- Solução para problema de quantização em dois níveis
 - necessária para uso em muitos dispositivos como impressão em dois níveis
 - Pontos pretos mais ou menos agrupados em um fundo branco dão a impressão de diferentes tons de cinza

Dithering

- A operação de *dithering* se fundamenta em uma característica da visão humana pela qual o olho integra os estímulos luminosos recebidos dentro de um certo ângulo sólido
- Percebemos, assim, intensidades que não necessariamente existem na imagem, resultantes do processo de integração dada pela média das intensidades de cor nas vizinhanças de cada elemento da imagem contidos no ângulo sólido

Dithering

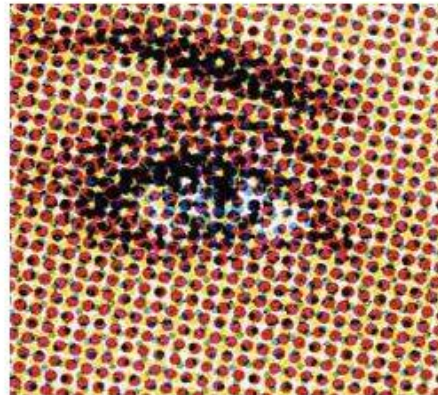
- Aplicações:
 - Um processo semelhante (na ideia) é aplicado em impressão de imagens coloridas: Offset



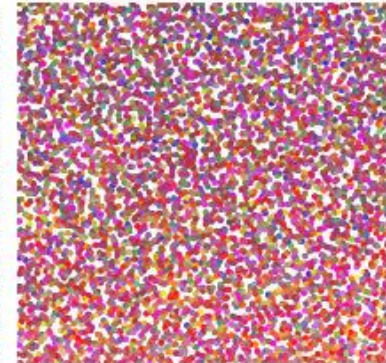
Roseta



Retícula Linear



Retícula Estocástica



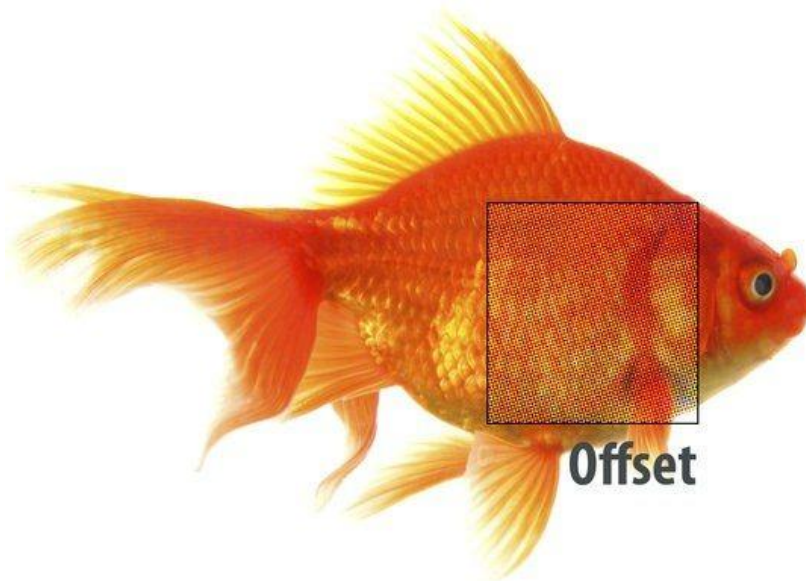
Pontilhamento

Fonte: <https://velhobit.com.br/design/vamos-falar-sobre-impressao.html>

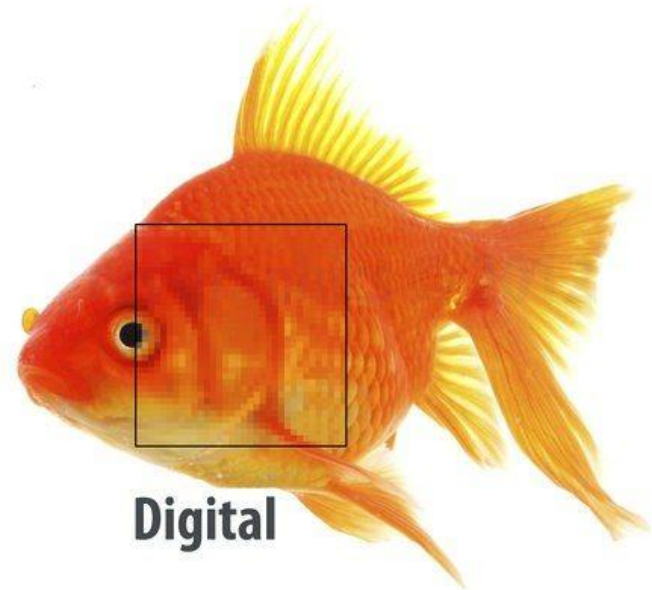
Dithering

- Aplicações:
 - Impressão digital vs Impressão Offset

Offset vs Digital Printing



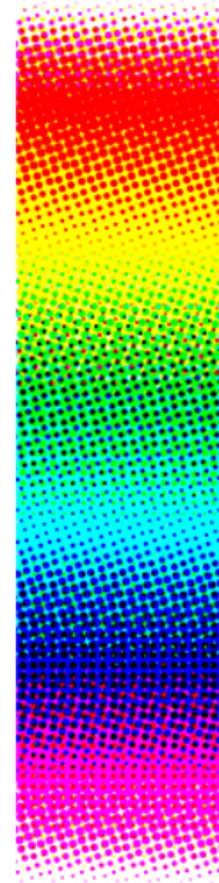
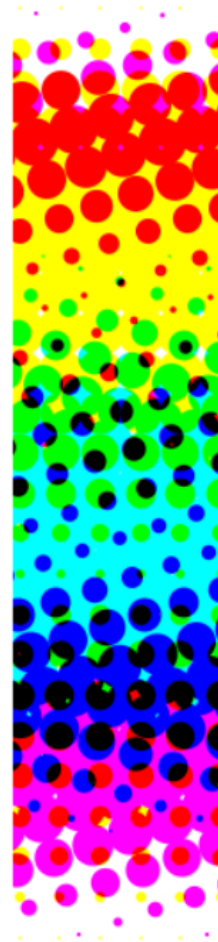
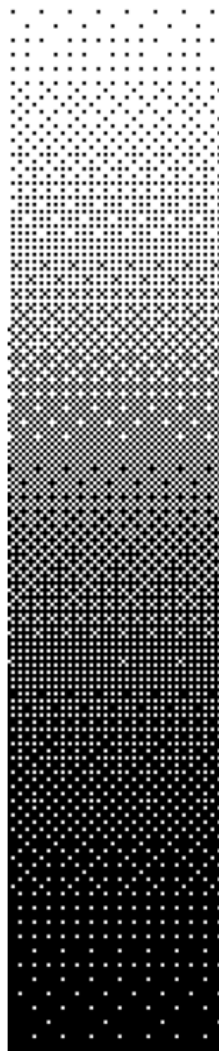
Offset



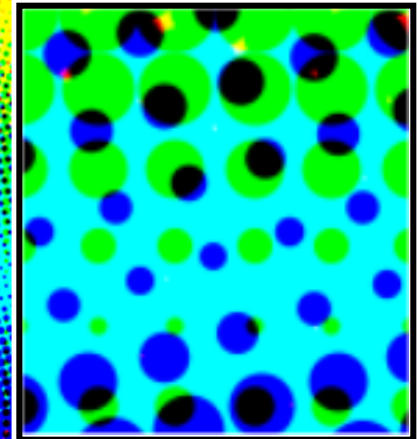
Digital

Dithering vs Halftoning

Dithering



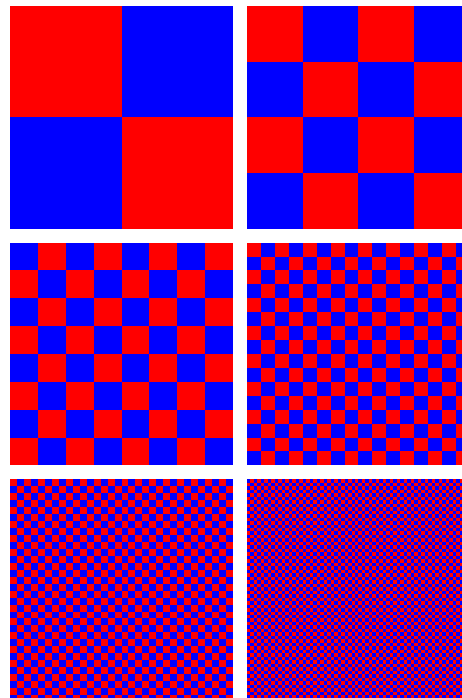
Halftoning



As cores se
sobrepõem e
se mesclam.

Dithering – Como?

- Dithering é o processo de agrupamento de pixels, utilizando duas cores para criar a ilusão de que há uma terceira cor presente



Dithering

Resolução e Percepção

- A resolução perceptual é medida pela *acuidade visual*
 - Capacidade do olho humano em detectar detalhes ao observar uma cena
 - O olho humano tem um ângulo de 150 graus na horizontal e 120 graus na vertical
 - O olho não distingue detalhes separados por um ângulo menor do que 1 minuto ($1/60$ grau)

Dithering

Resolução e Percepção

- A percepção de detalhes em uma imagem depende de três parâmetros:
 - Distância da imagem ao olho
 - ↑ distância $\Rightarrow$ ↓ detalhes
 - Densidade de resolução
 - Dots per inch*
 - Abertura do olho
 - ↓ abertura $\Rightarrow$ ↓ detalhes

Dithering

- Algoritmos de Dithering
 - Abordagem Estatística
 - Abordagem Determinística

Dithering Algoritmos

- Dithering por Modulação Aleatória

Color_Intensity = $f[x][y]$;

if (Color_intensity < threshold)

 Color_Intensity = 0;

else

 Color_Intensity = 1;

- Variação

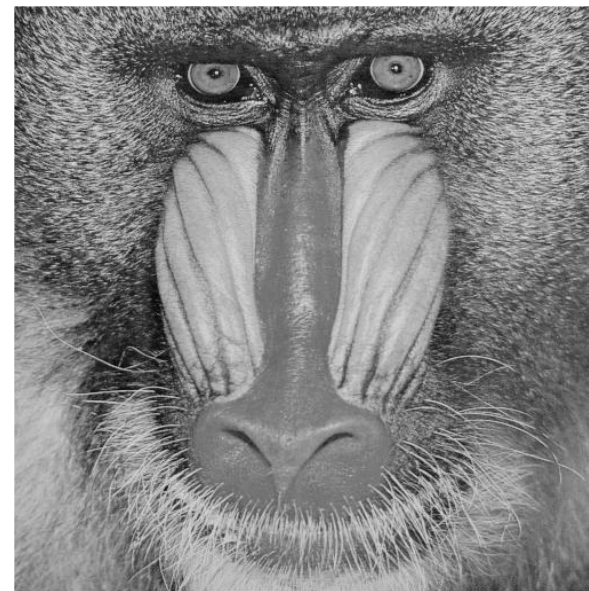
Color_Intensity = $f[x][y]$ + random;

Problema: Aleatoriedade

Dithering por Modulação Aleatória



```
I = imread('baboon.bmp');  
c=150; % Ponto de corte  
d=10;  
I = I + d*randn(1);  
I = (I > c);  
imshow(I);
```



Dithering por Modulação Aleatória

$d = 10$



$d = 20$



$d = 30$



Dithering

Algoritmos

- Dithering Periódico

- A imagem é particionada em blocos contíguos de ordem N
- Cada bloco da imagem é comparado com uma máscara de dithering
 - Se o valor do elemento da imagem for menor que o da máscara, torna-se preto, caso contrário, torna-se branco
 - Os tons dos pixels no bloco devem ser normalizados para o valor máximo da máscara

Dithering Periódico



Máscara de
Dithering →

Bayer ordem 2

1	3
4	2

Bayer ordem 4

1	9	3	11
13	5	15	7
4	12	2	10
16	8	14	6

Dithering Algoritmos

- Difusão de Erro

- *Erro* = diferença cumulativa entre os atuais valores dos pixels na imagem e seus verdadeiros valores se eles estivessem em suas cores corretas
- O processo começa no primeiro pixel. O algoritmo encontra a cor na nova paleta que mais se aproxima da cor real do pixel. Compara as duas cores e salva a diferença como um erro inicial. Aplica a cor encontrada no pixel

Dithering

Algoritmos

- Difusão de Erro

- O segundo pixel é lido e a cor mais próxima é encontrada. O algoritmo calcula a soma da cor com o valor do erro inicial
- O pixel ganha a cor mais próxima dessa soma
- Qualquer diferença entre a cor aplicada e a soma se torna o novo valor de erro
- O erro é desprezado no final de cada linha

Floyd-Steinberg – Dispersão de Erro

- Dispersão de erro
 - Achar a cor disponível mais próxima de cada ponto da imagem
 - Calcular a diferença entre o valor da imagem e da cor que você tem
 - Divida os erros e distribua eles através dos pixels vizinhos que não foram visitados ainda
 - Diferença para Difusão de erro
 - Depois adicione os erros e distribua-os
 - Limita os valores da escala se precisar e continue como os acima

Floyd-Steinberg – Dispersão de Erro

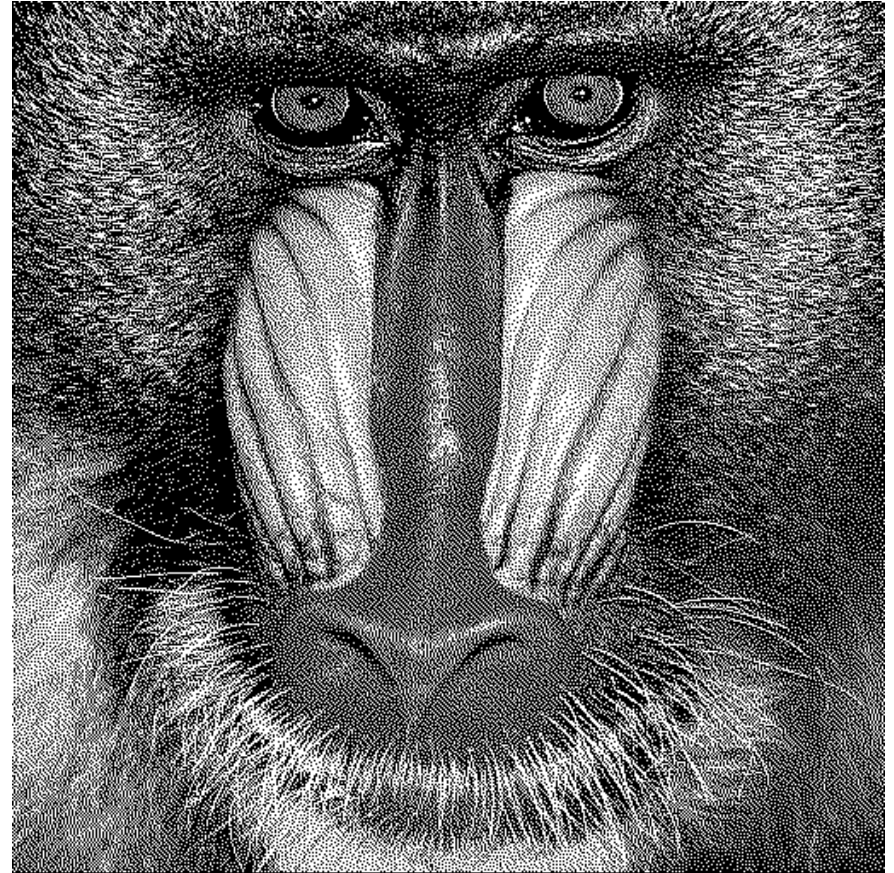
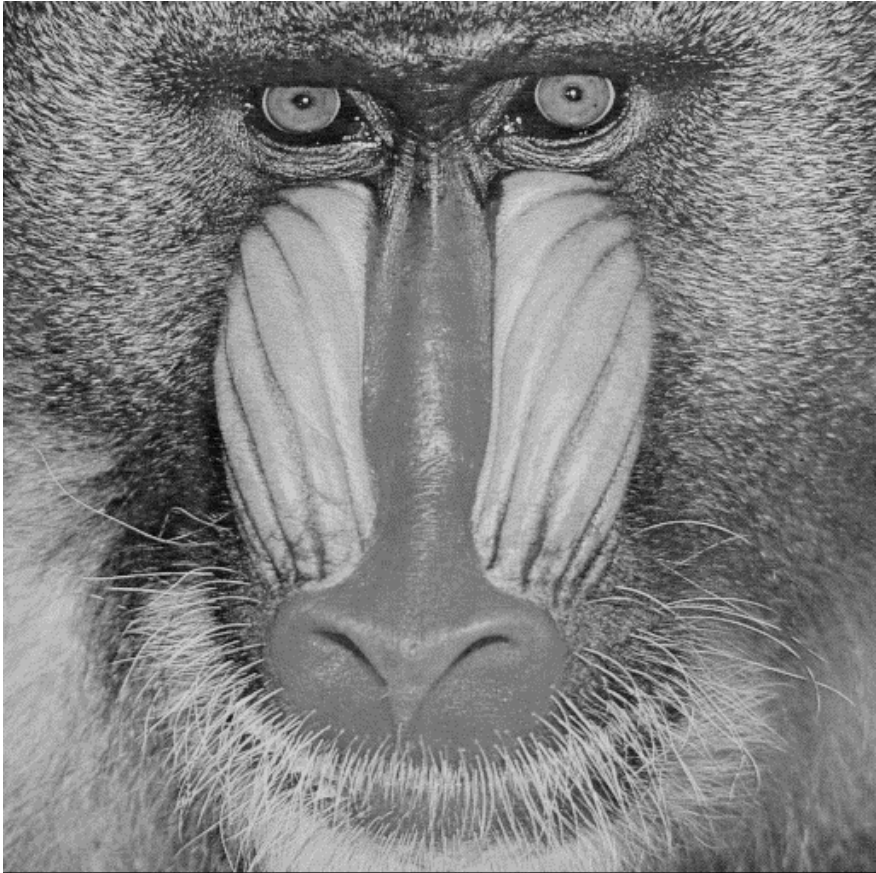
- Máscara:

	X	7/16
3/16	5/16	1/16

O 'X' representa o pixel em análise e os números são pesos, representam a proporção do erro distribuído no pixel da posição.

- Não considera o elemento anterior já que não faz sentido distribuir o erro para o passado

Floyd-Steinberg – Dispersão de Erro



Stucki – Dispersão de Erro

- Erro = diferença cumulativa entre os atuais valores dos pixels na imagem e seus verdadeiros valores se eles estivessem em suas cores corretas
- O processo começa no primeiro pixel. O algoritmo encontra o bit (preto ou branco) que mais se aproxima da cor real do pixel. Compara as duas cores e salva a diferença como um erro inicial. Aplica a cor encontrada no pixel

Stucki – Dispersão de Erro

- O erro é distribuído entre os pixels da vizinhança de acordo com a máscara
- O segundo pixel é lido. O algoritmo calcula a soma da cor com o valor do erro inicial. O pixel ganha a cor mais próxima dessa soma. Qualquer diferença entre a cor aplicada e a soma se torna o novo valor de erro
- Vizinhança mais Ampla e peso maior

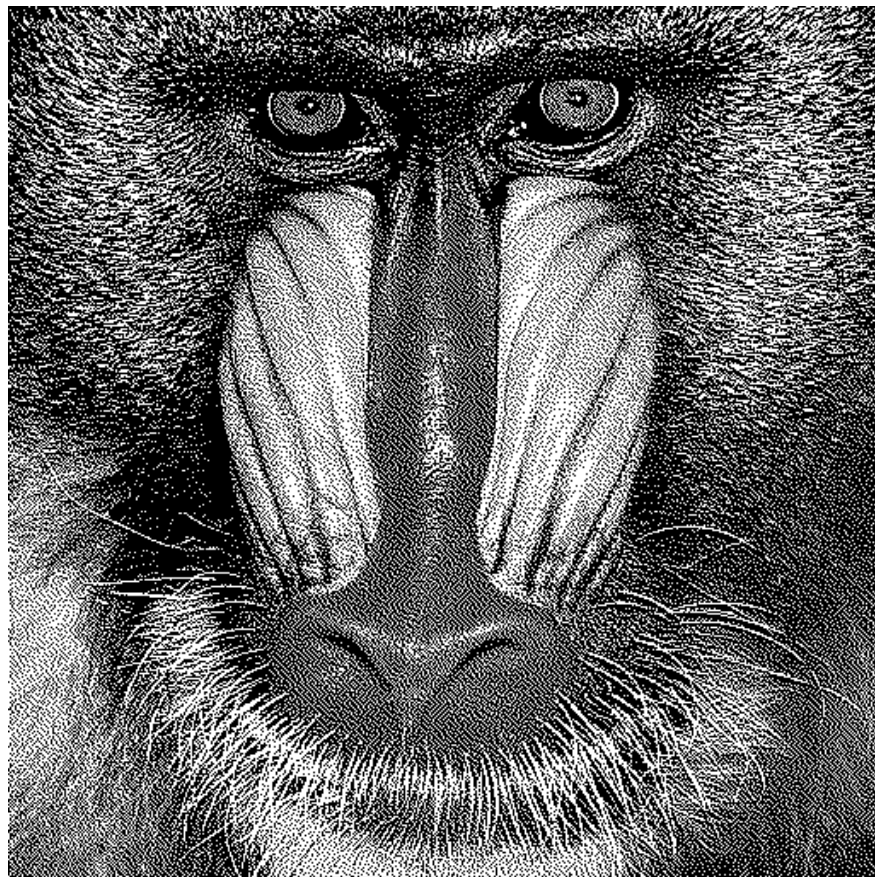
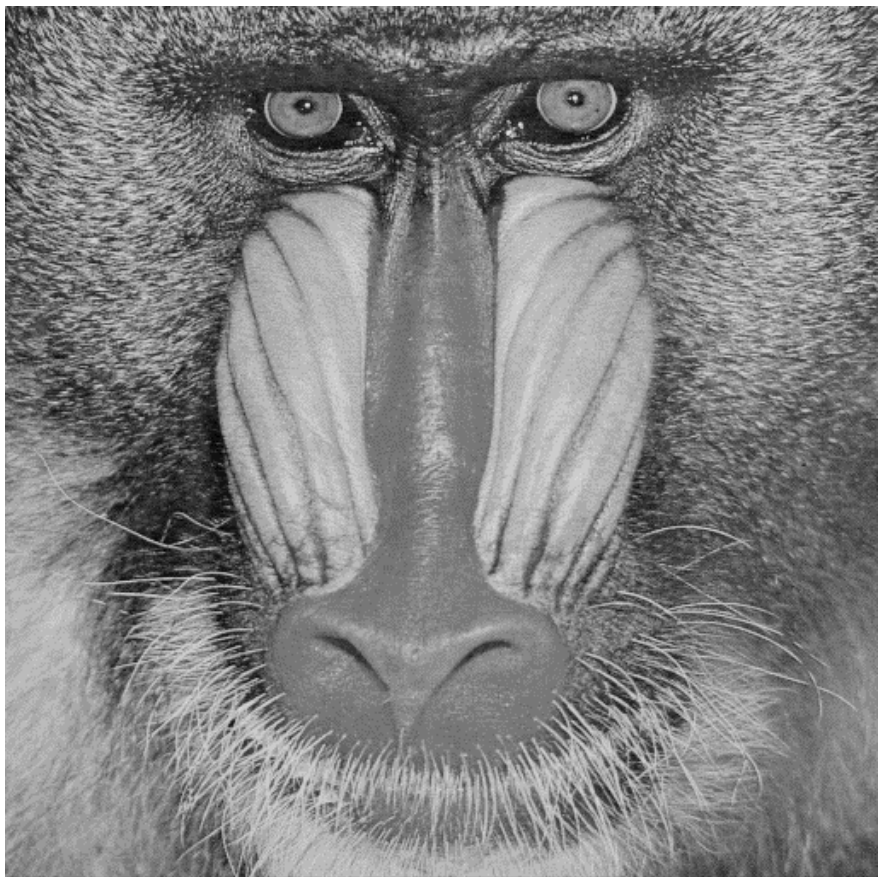
Stucki – Dispersão de Erro

- Máscara do Algoritmo de Stucki

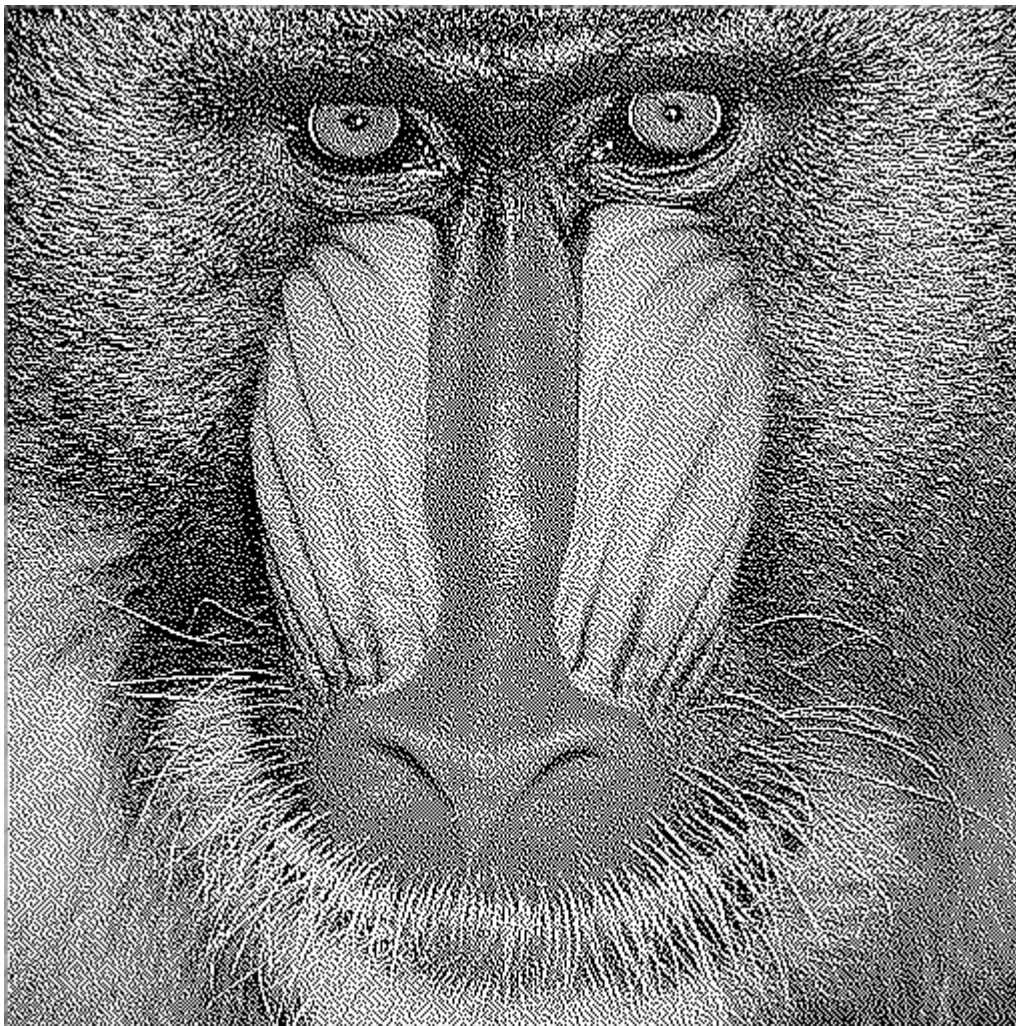
$$\frac{1}{42} \times$$

		●	8	4
2	4	8	4	2
1	2	4	2	1

Stucki – Dispersão de Erro



Stucki – Dispersão de Erro



Burkes – Dispersão de Erro

- Este método é uma simplificação do de Stucki com a linha de baixo removida
- Filtro:

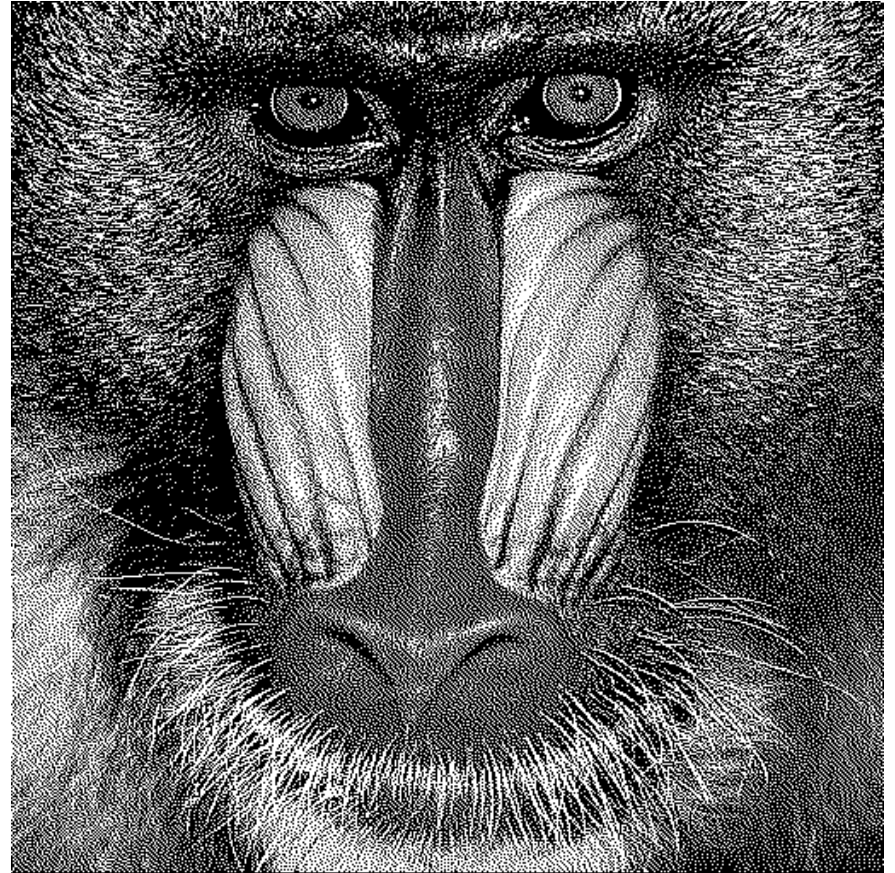
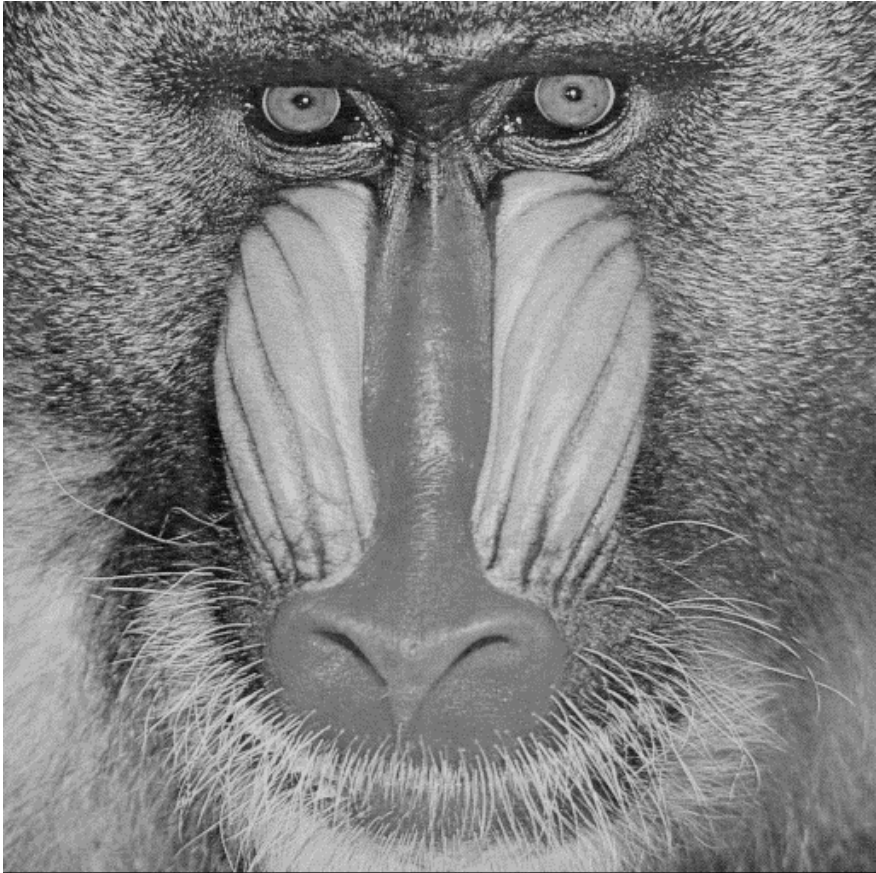
		X	8/32	4/32
2/32	4/32	8/32	4/32	2/32

A principal melhoria é que o divisor é agora 32 e o número dos vizinhos foi reduzido a sete

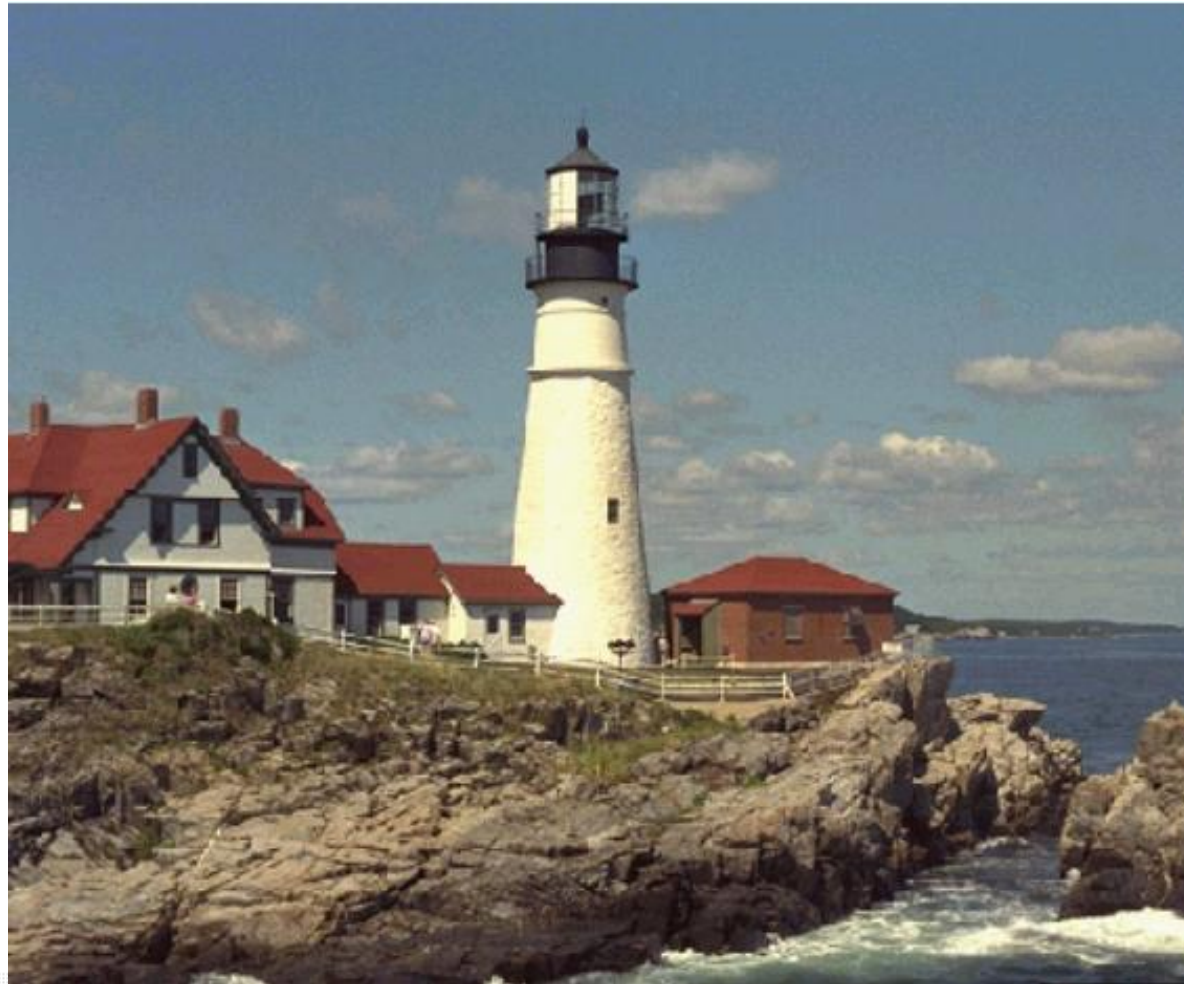
Burkes – Dispersão de Erro

- Esta máscara distribui os pesos de forma balanceada entre os pontos, dando a mesma ênfase para os pixels imediatamente a direita e abaixo, e proporcionalmente aos mais afastados

Burkes – Dispersão de Erro



Dithering Exemplo



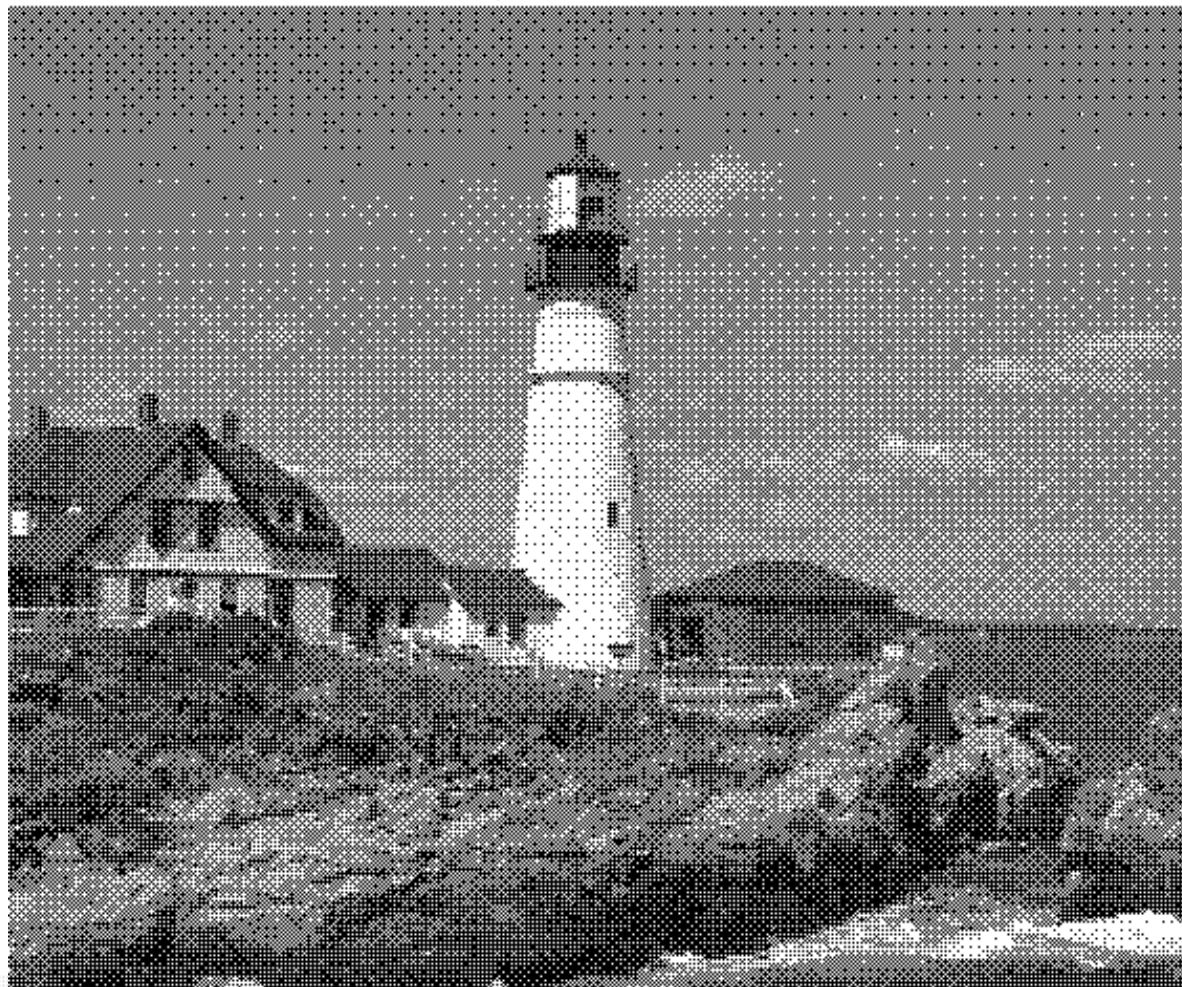
Dithering

Redução de
cores sem
Dithering



Dithering

Redução de
cores com
Dithering



Aplicações de Dithering

- Embora, hoje em dia, o uso de *dithering* seja menos comum (porque há mais usabilidade de imagens com 16 milhões de cores), ainda temos algumas importantes aplicações:
 - 1- Uso em e-Books
 - e-Books, usualmente, são menos custosos se leem livros digitais apenas em preto e branco. Isso exige que imagens e ilustrações, para terem melhor qualidade visual, passem por um processo de *dithering*
 - 2- Formatos de armazenamento
 - O formato GIF trabalha apenas com imagens com 256 cores. Para melhorar a qualidade delas, usa-se *dithering*

Aplicações de Dithering

- Exemplo de imagem com 16 milhões de cores:



Imagem com 127.281 cores

Aplicações de Dithering

- Exemplo de imagem GIF (256 cores):



Aplicações de Dithering

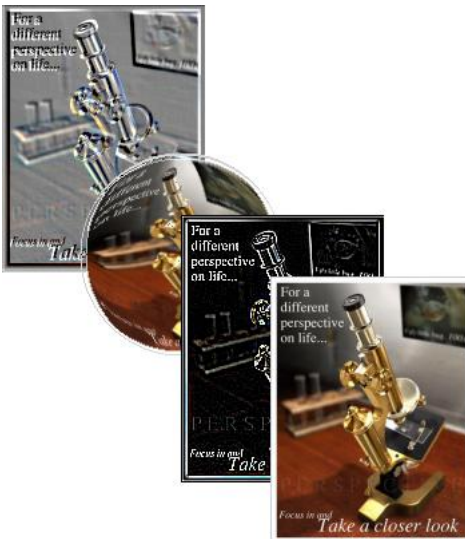
- Exemplo de imagem GIF (256 cores):

Em zoom...



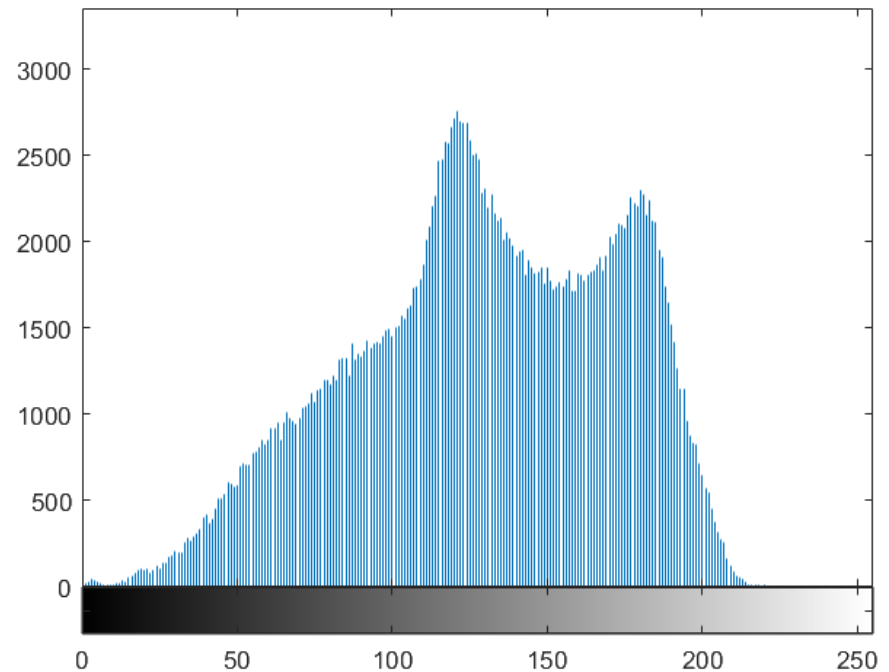
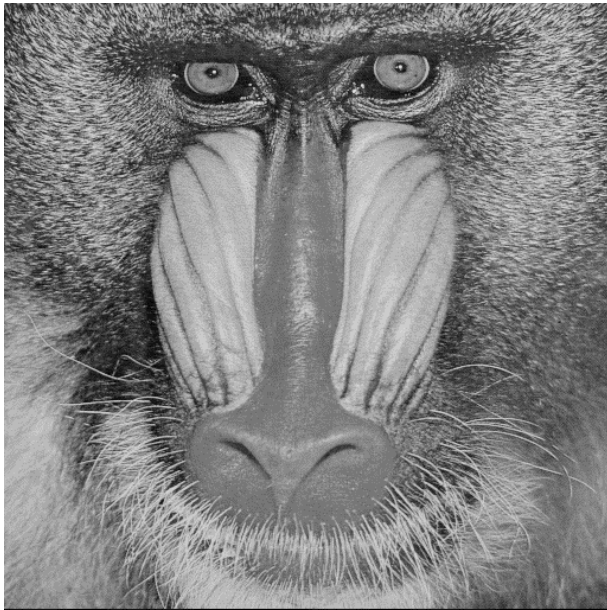
Histograma

Carlos Alexandre Barros de Mello
CIn/UFPE



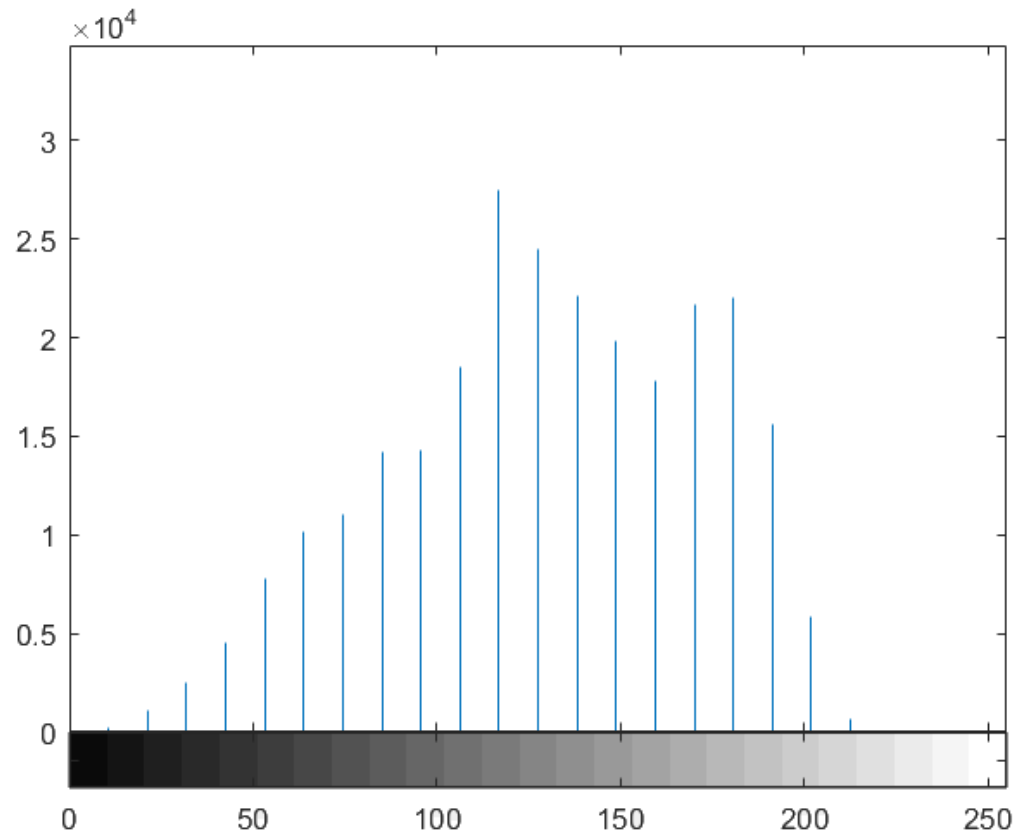
Histograma

- O **Histograma** de uma imagem provê uma descrição global da aparência da imagem em termos de distribuição de cores

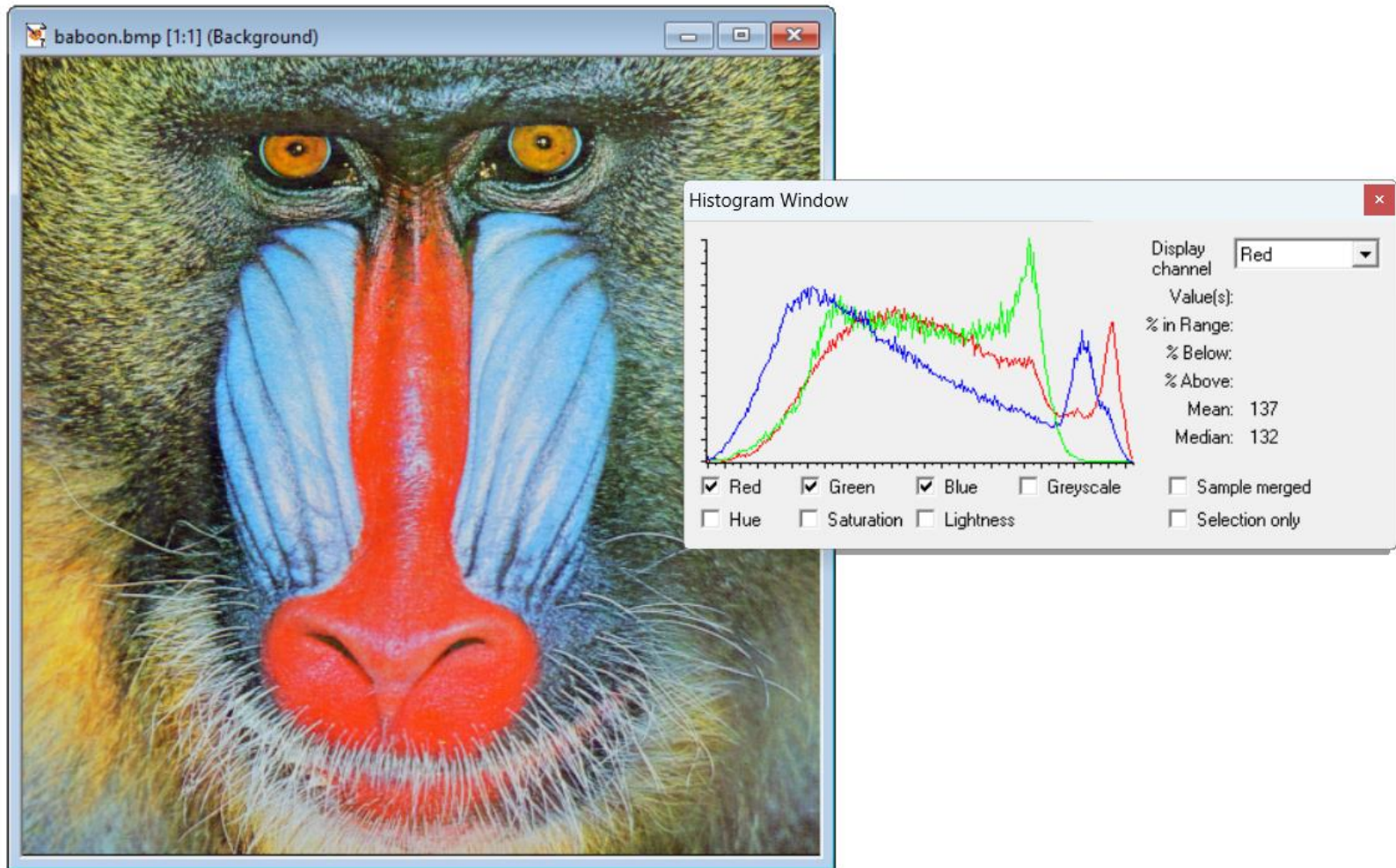


Histograma

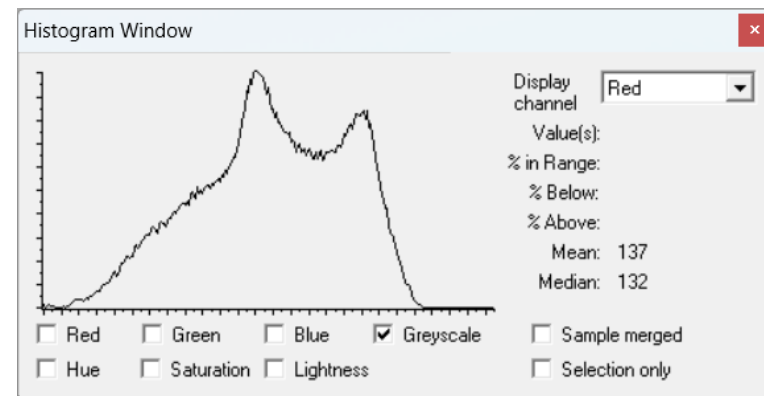
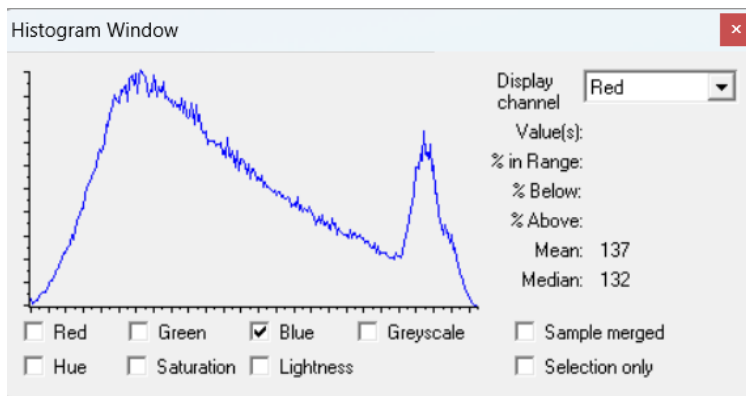
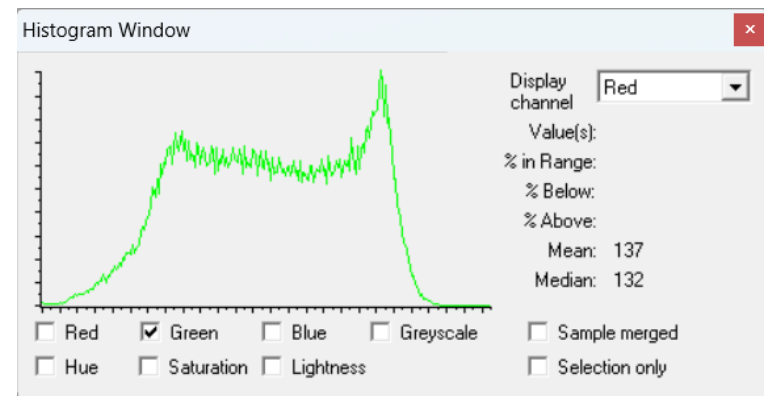
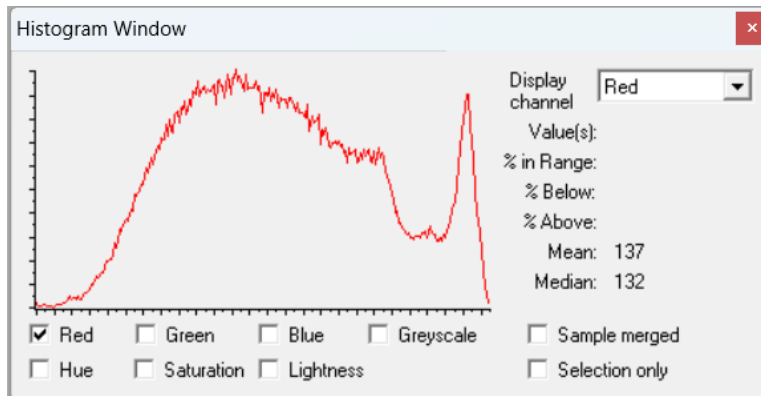
Em *bins* (no caso, 25)... Divide o histograma em 25 faixas de valores



Histograma

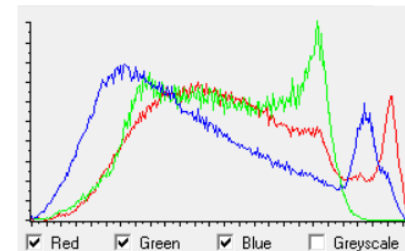
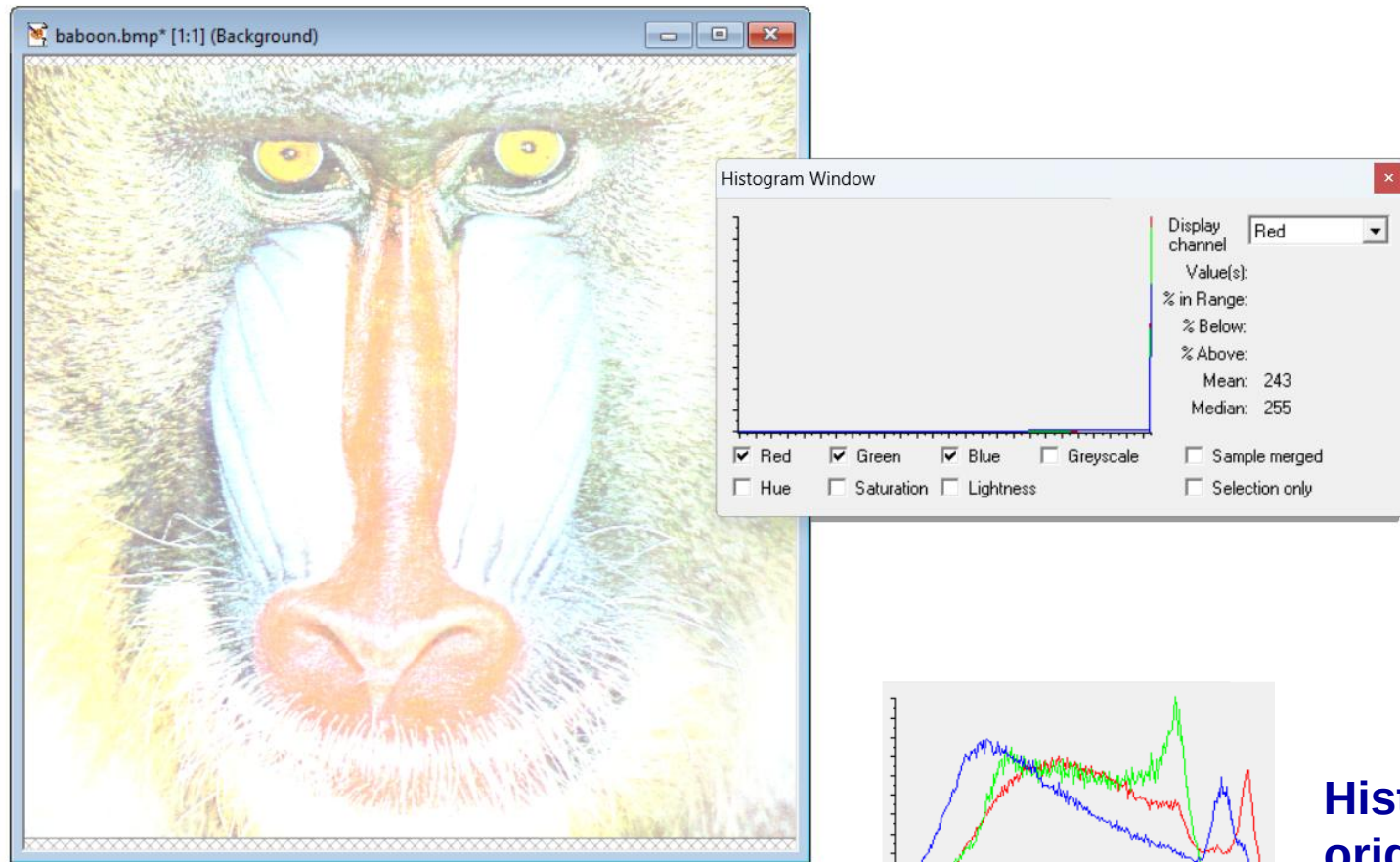


Histograma



Histograma

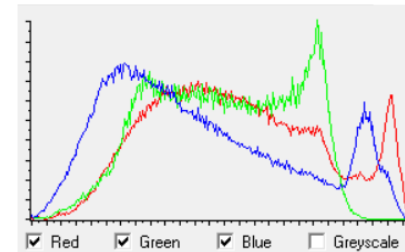
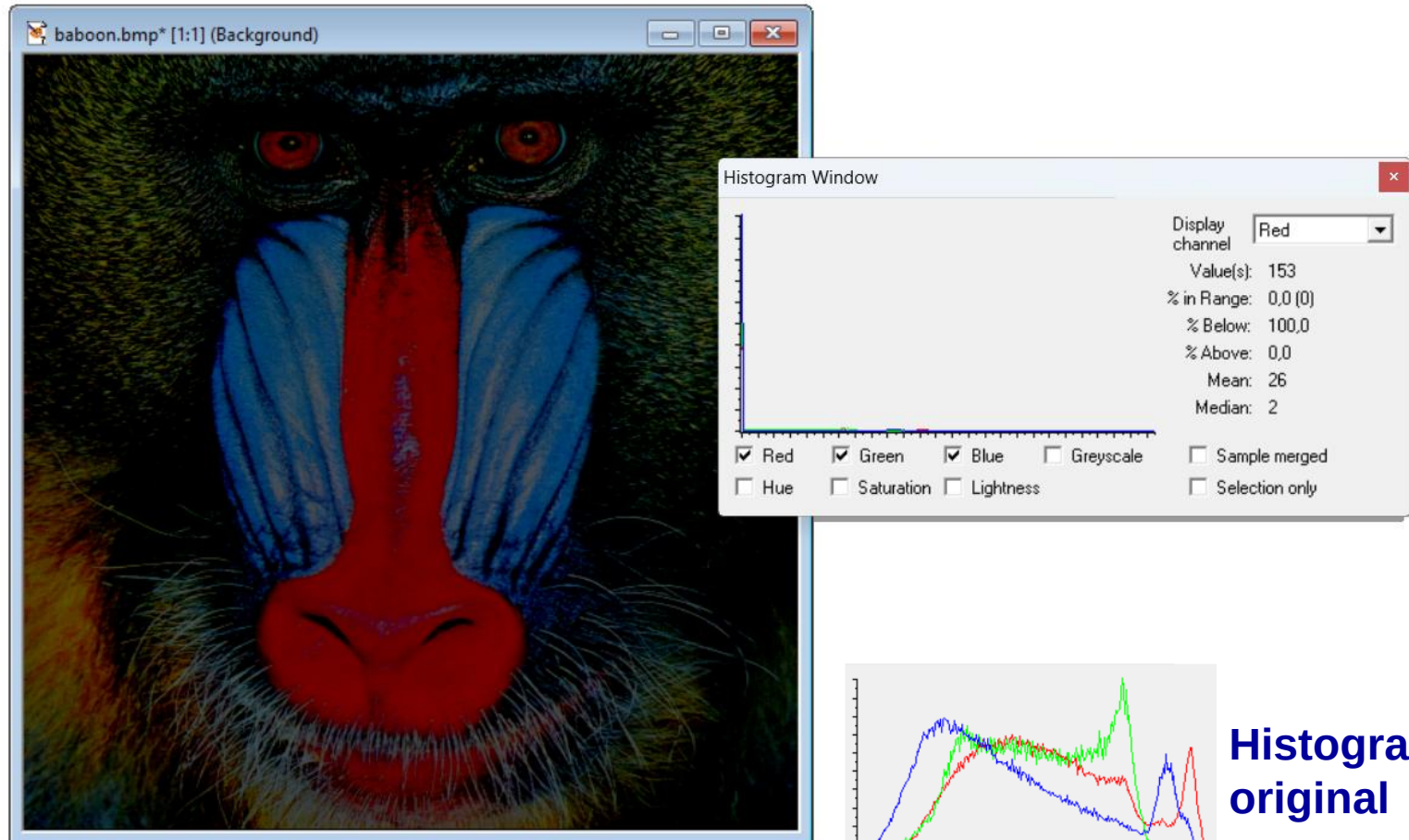
Imagem muito clara: histograma próximo do branco...



Histograma original

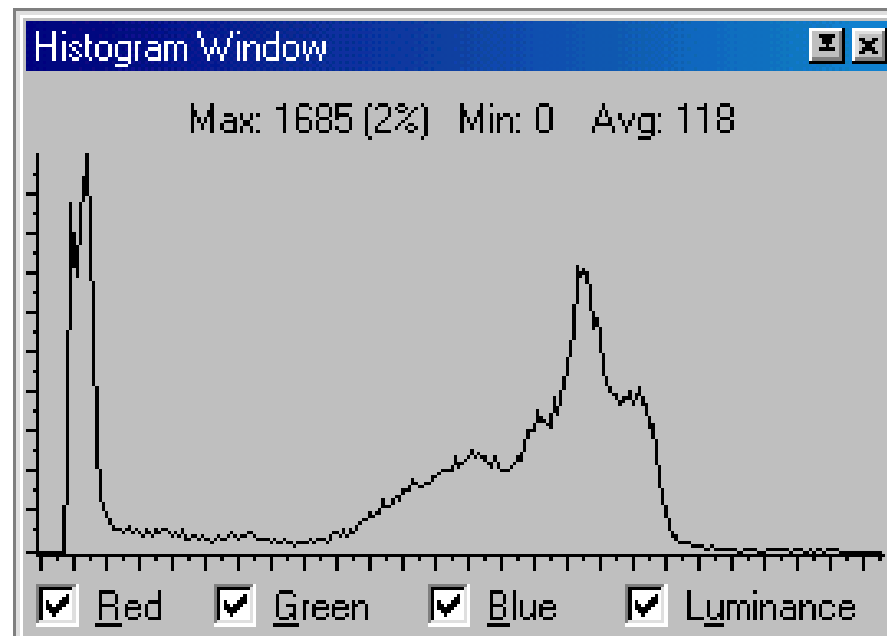
Histograma

Imagem muito escura: histograma próximo do preto...



Histograma original

Histograma e Contraste



Contraste: nível de separação entre as cores

Histograma e Contraste

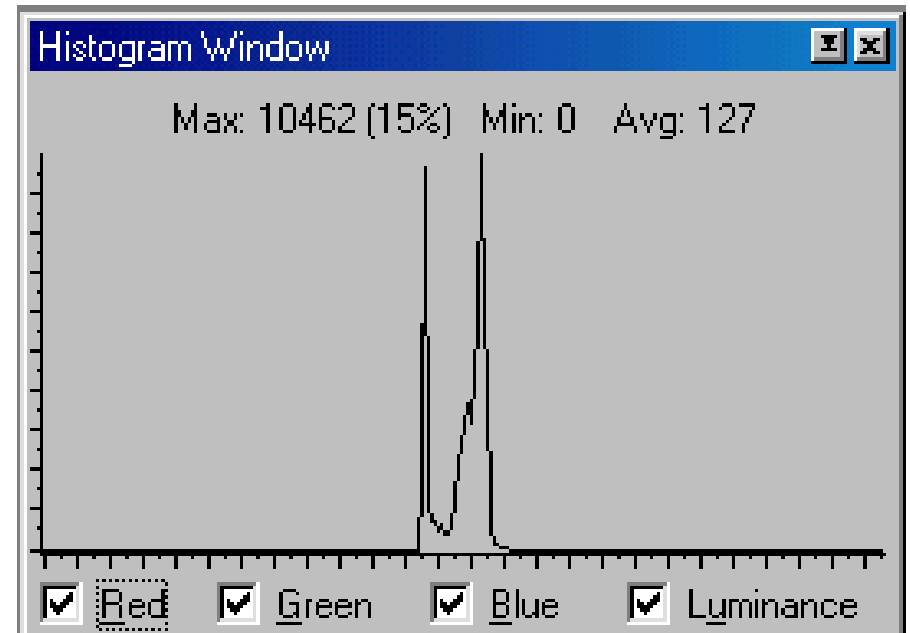


Imagem de (mais) Baixo Contraste

Histograma e Contraste

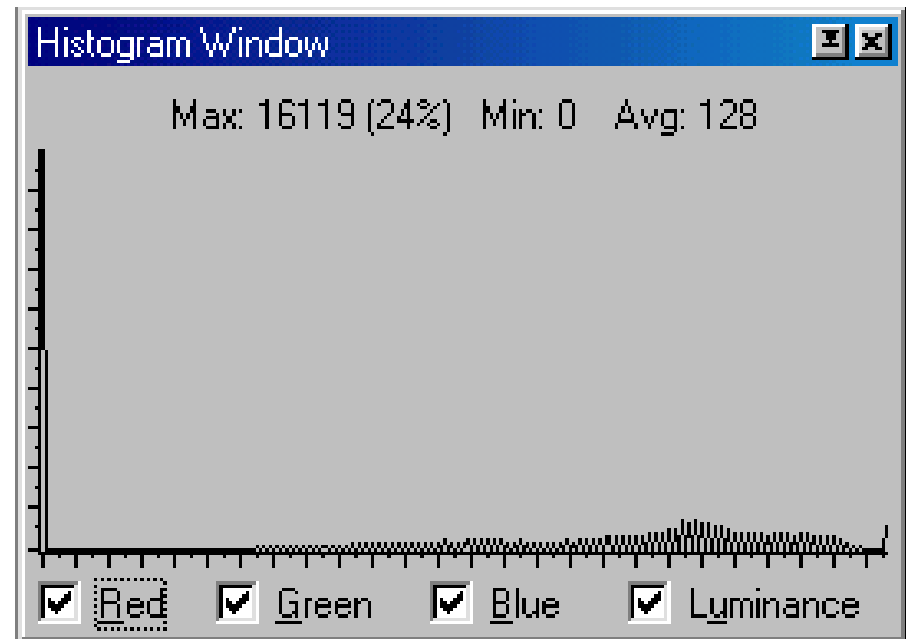


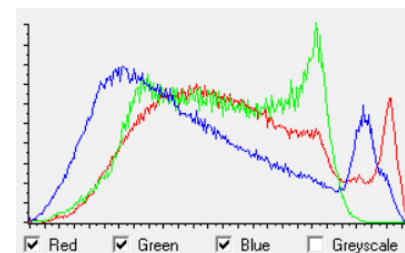
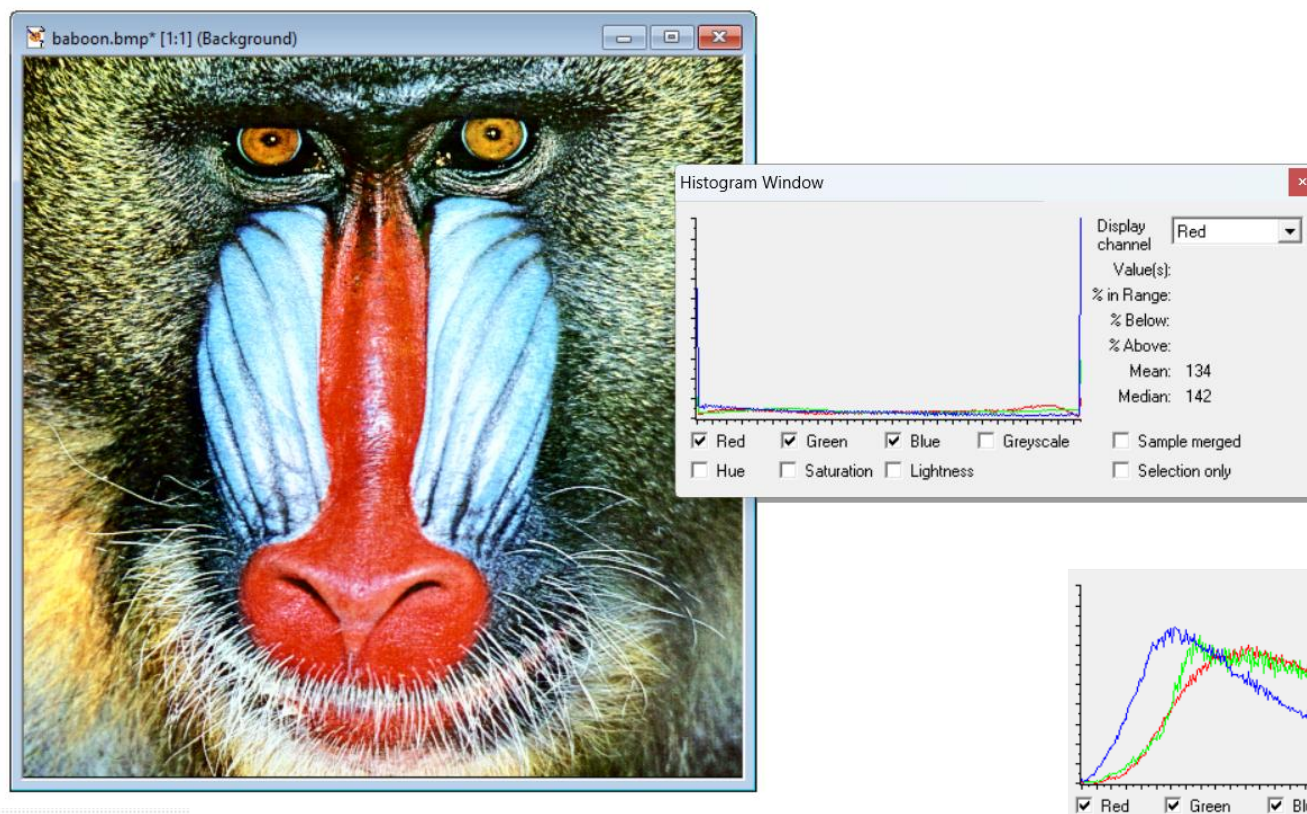
Imagem de (mais) Alto Contraste

Histograma

Operações

Equalização

Aproximação para um
histograma uniforme



Histograma

Operações

- Equalização:
 - torna o pixel mais escuro **mais próximo** do preto e o mais claro **mais próximo** do branco
 - Distribui o restante das cores mais uniformemente nesse intervalo

Histograma

Operações

- Equalização:
 - Operação simples:
 - Para cada nível de brilho j na imagem original (e seu histograma), um novo valor k é atribuído, onde:

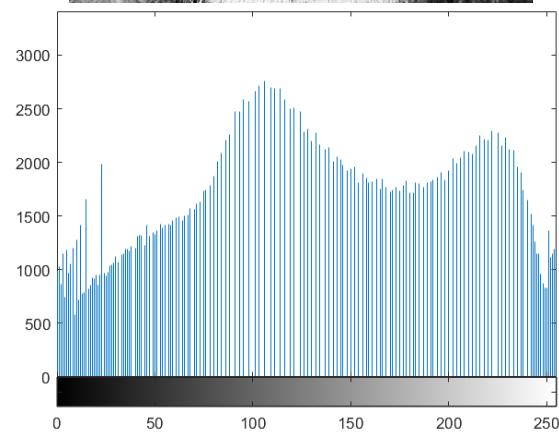
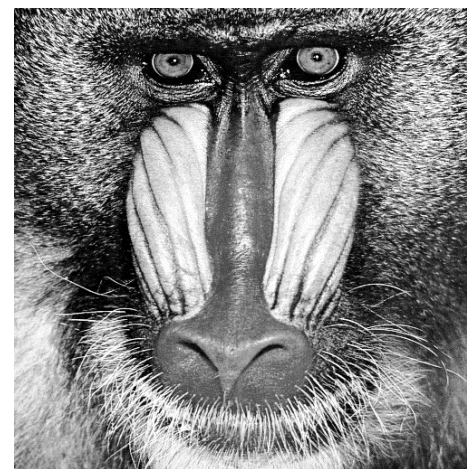
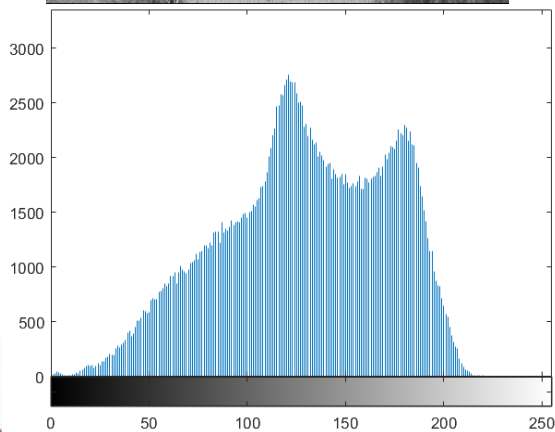
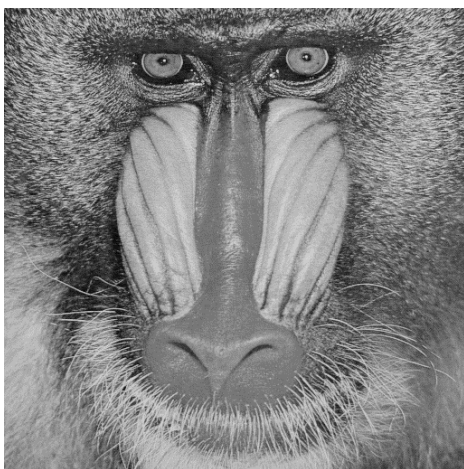
$$k = \sum_{i=0}^j \frac{N_i}{T}$$

- onde T é o número de pixels na imagem e N_i é o total de pixels na posição i do histograma

Histograma

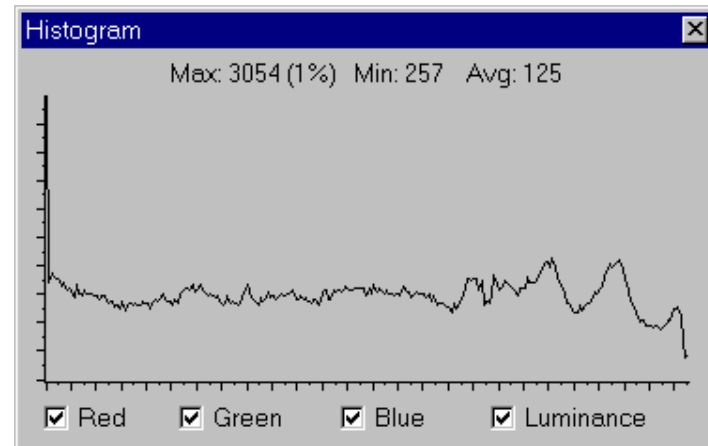
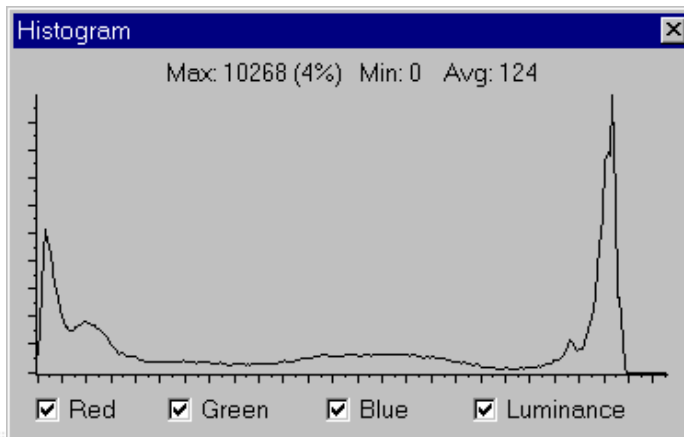
Operações

- Equalização:



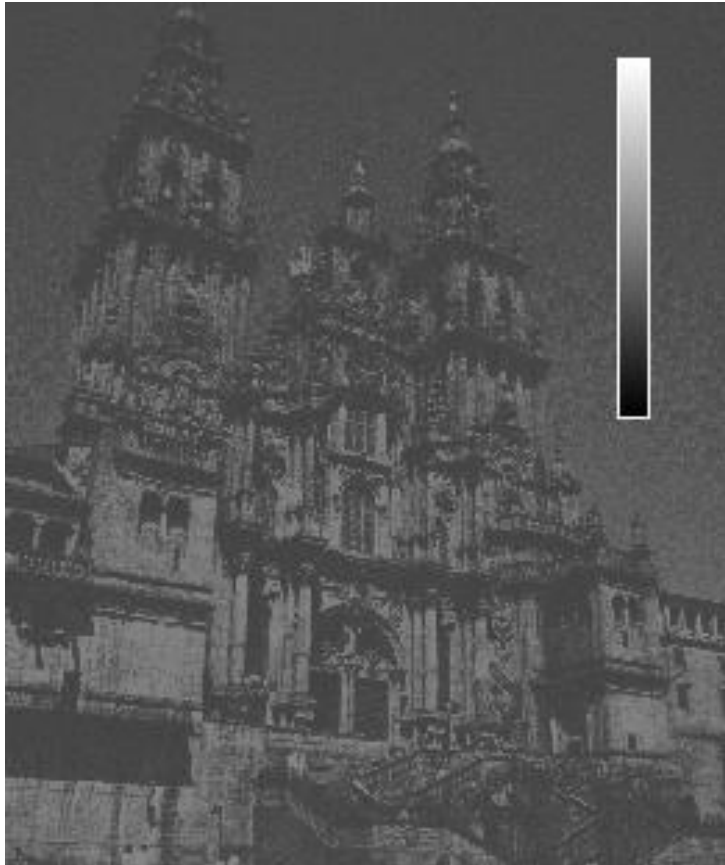
Histograma

Equalização - Exemplos



Histograma

Equalização - Exemplos



Histograma

Equalização - Exemplos

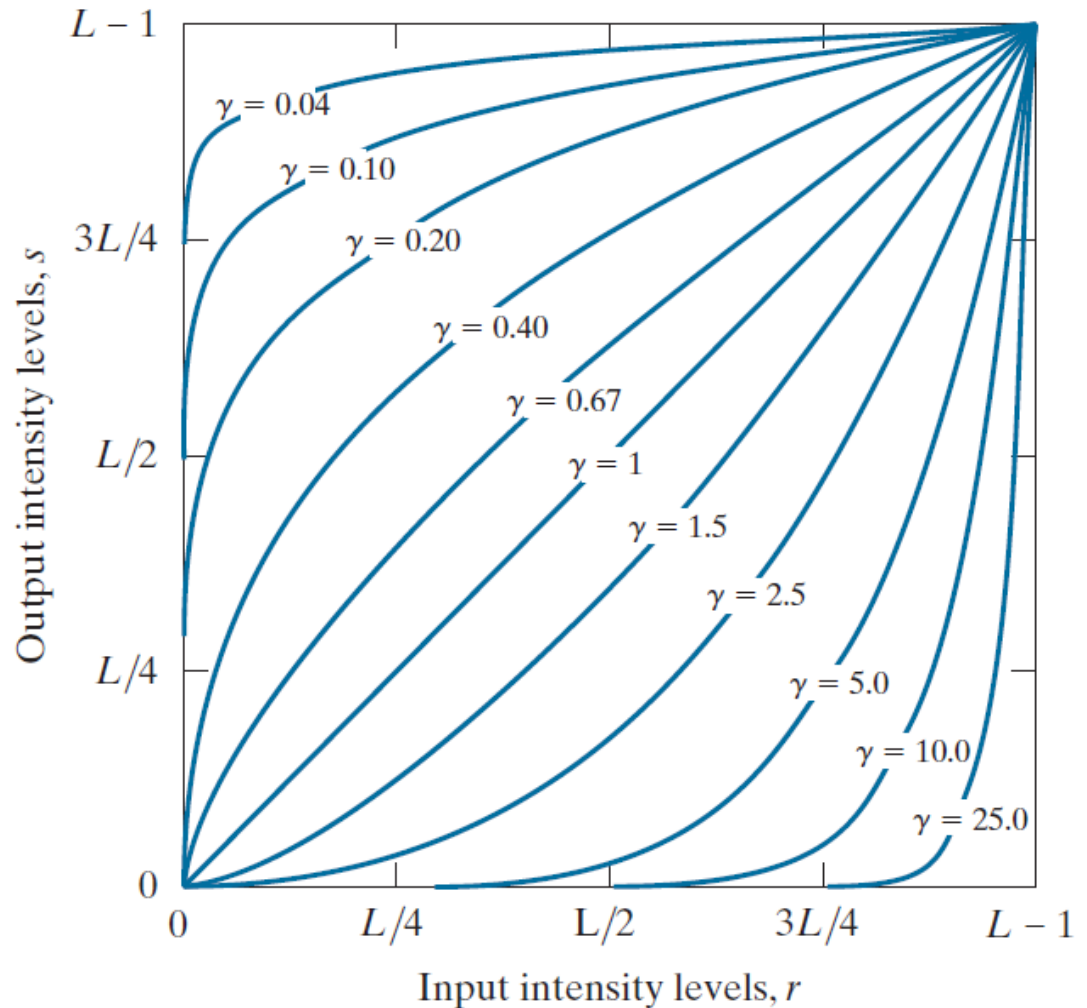


Correção Gamma

- Transformação expressa por:
 - $Im_{out} = c.(Im_{in})^\gamma$
 - onde Im_{in} é a imagem de entrada em tons de cinza, normalizada (valores entre 0 e 1)
 - c , geralmente, é igual a 1
 - γ é o fator de correção, considerando que:
 - $\gamma < 1$: Expande os valores de intensidade mais baixos, clareando áreas escuras
 - $\gamma > 1$: Comprime os valores de intensidade mais baixos, deixando a imagem mais escura
 - $\gamma = 1$: Com $c = 1$, não altera a imagem
 - É uma transformação não linear (se $\gamma \neq 1$)

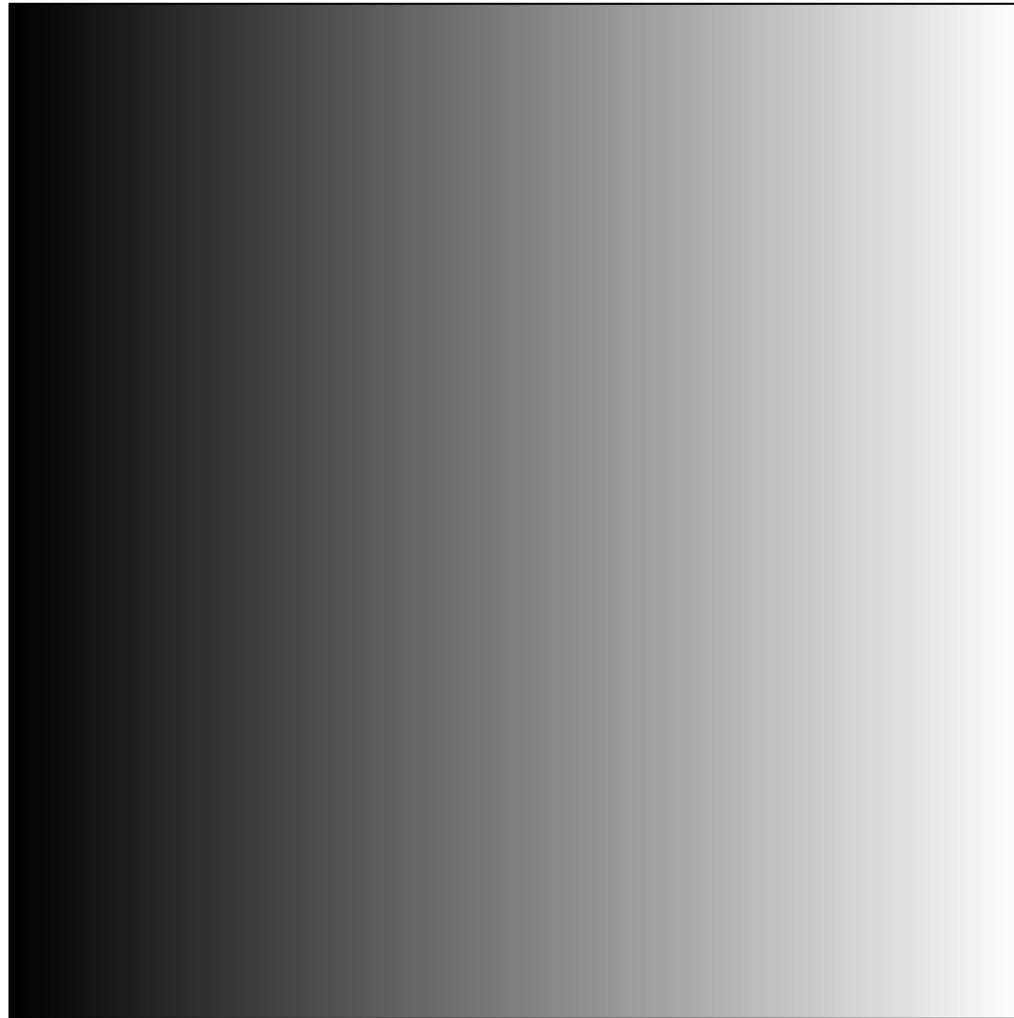
Correção Gamma

Plotagem da transformação Gamma, variando o parâmetro γ , considerando $c = 1$, e ajustando as curvas para a mesma escala para facilitar a comparação visual.



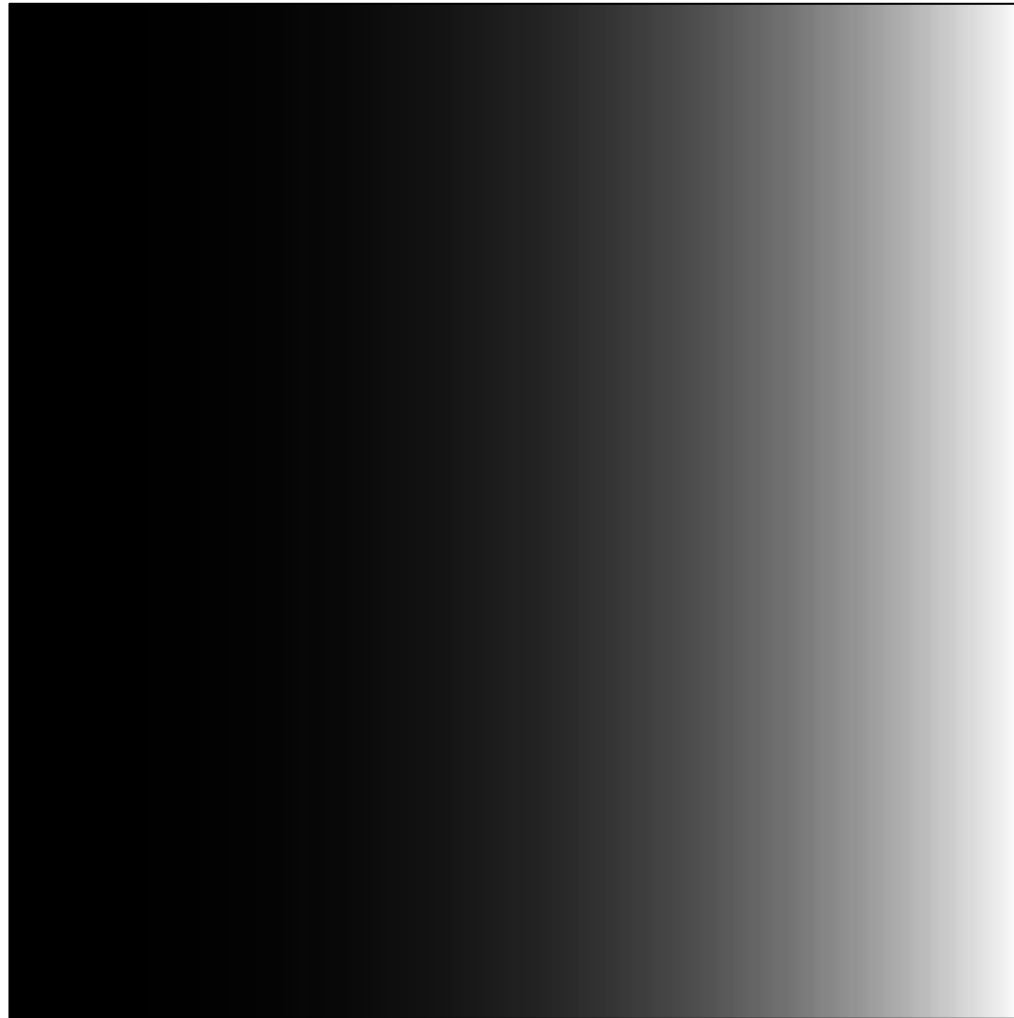
Correção Gamma

Imagem
Original



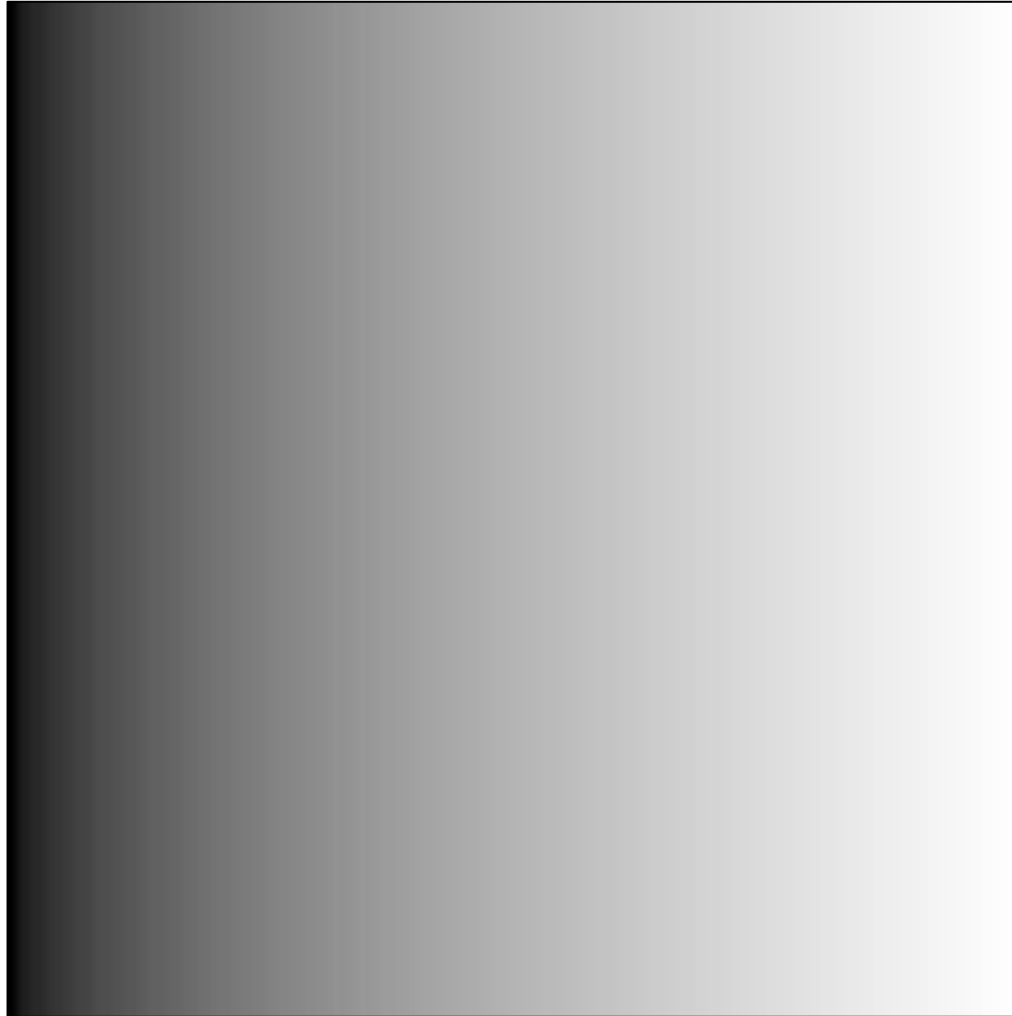
Correção Gamma

$$\gamma = 3$$

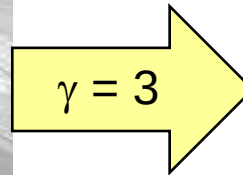


Correção Gamma

$$\gamma = 0,5$$



Correção Gamma



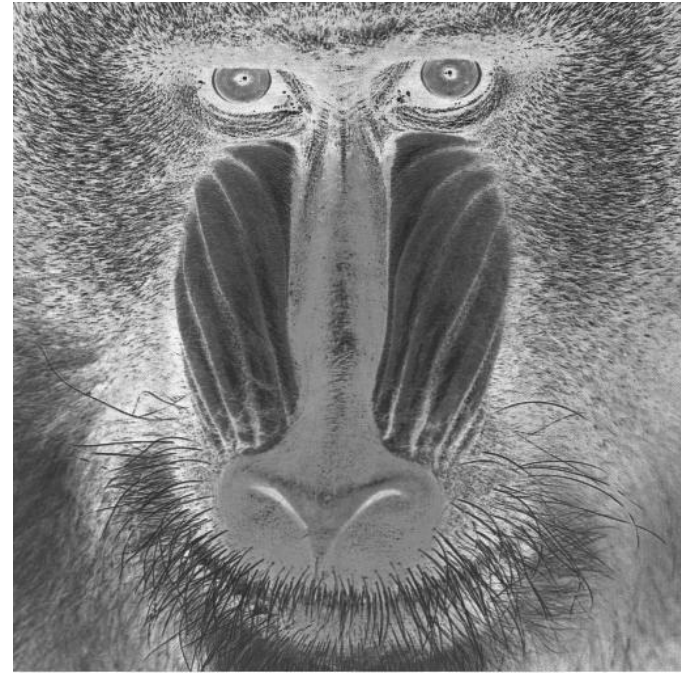
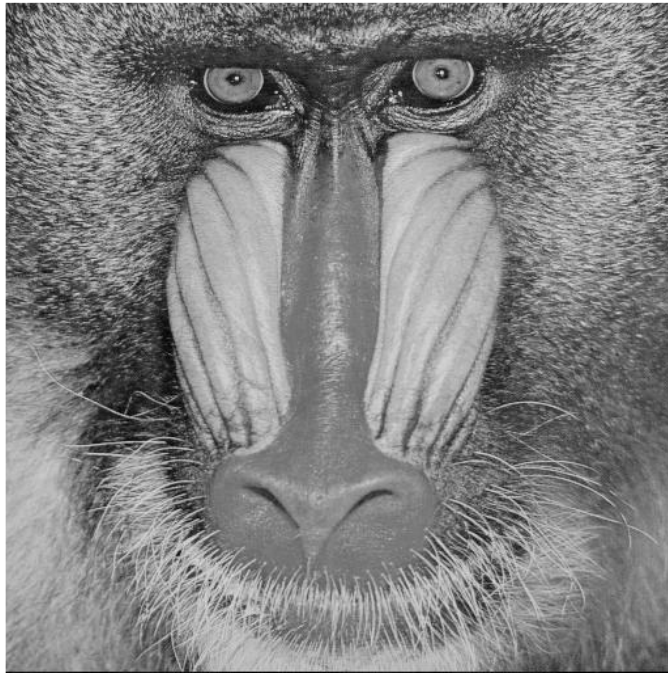
Técnicas de modificação de histograma

- São técnicas utilizadas para processar a imagem através da modificação do histograma
- Exemplos
 - Negativo (Complemento)
 - Binarização
 - Brilho
 - Expansão
 - Compressão

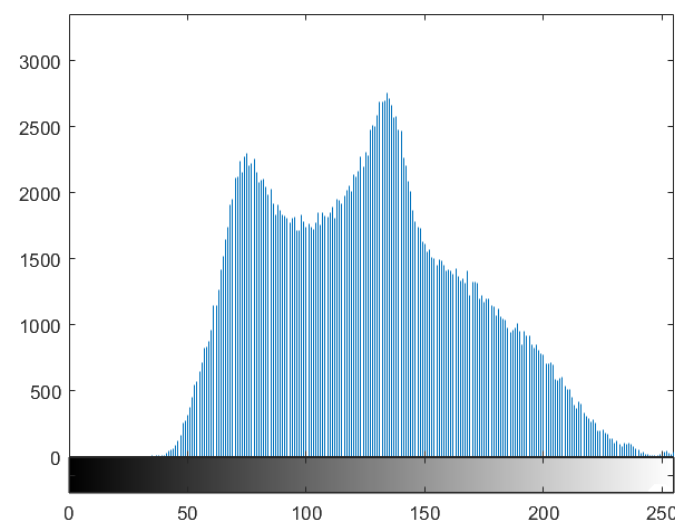
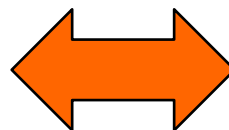
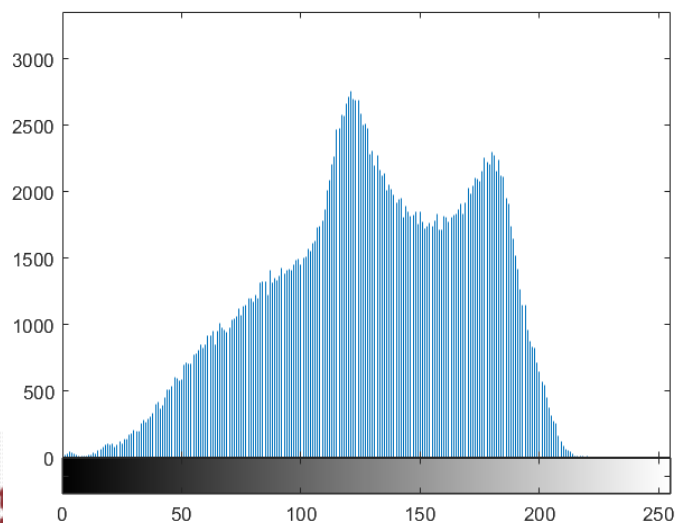
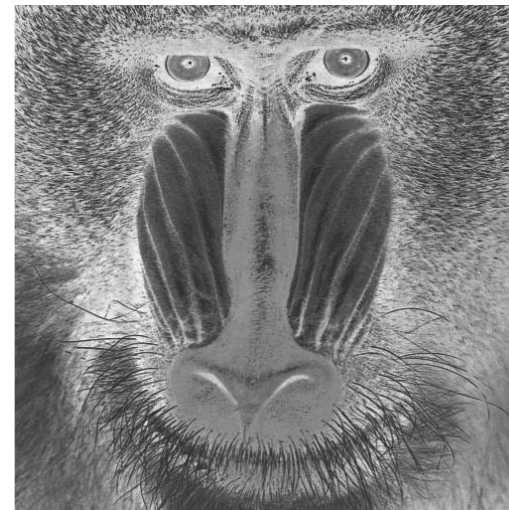
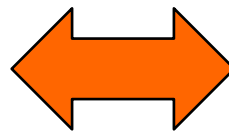
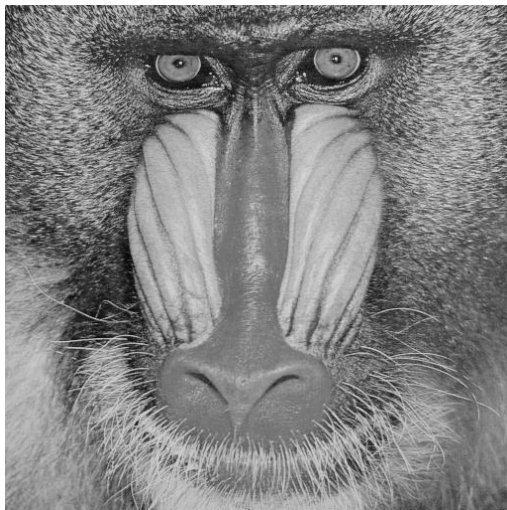
Complemento

I = Imagem em 256 tons de cinza

$$I_{\text{comp}} = 255 - I$$



Complemento



Complemento em Imagens Coloridas

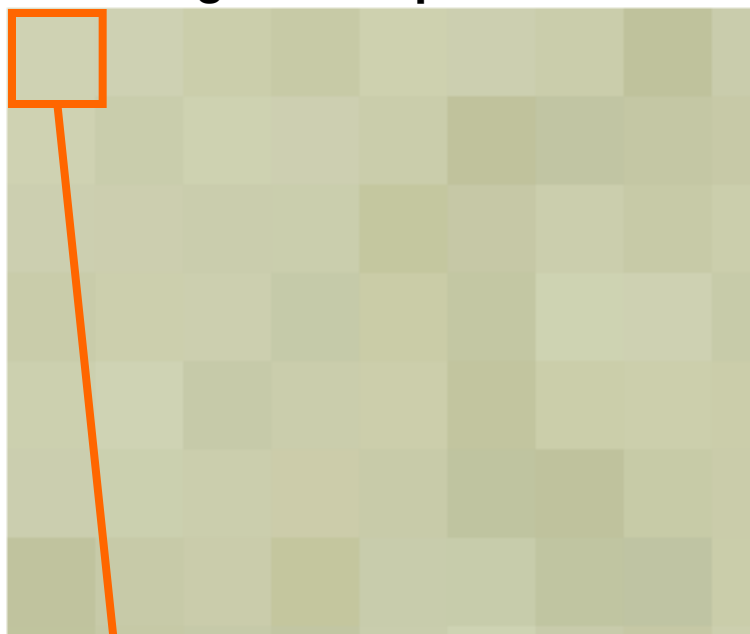


Complemento em Imagens Coloridas



Complemento em Imagens Coloridas

Imagem Complementar



(207, 210, 179)

Imagem Original



(48, 45, 76)

+

(255, 255, 255)

Binarização

- Consiste em separar o histograma de uma imagem em regiões classificando-as como uma classe (0 – preto) ou outra (1 – branco)
- Em imagens preto e branco
 - quando o pixel apresentar um tom de cinza mais próximo do preto, ele é convertido para preto
 - quando apresentar um tom de cinza mais próximo do branco, ele é convertido para branco

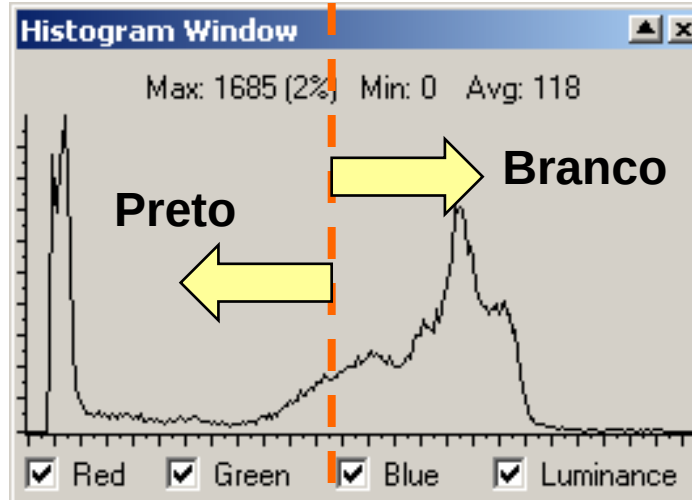
Binarização

- Algoritmo de Recorte

Se $\text{cor}(i) \leq 127$

Então $\text{cor}(i) = \text{Preto (0)}$

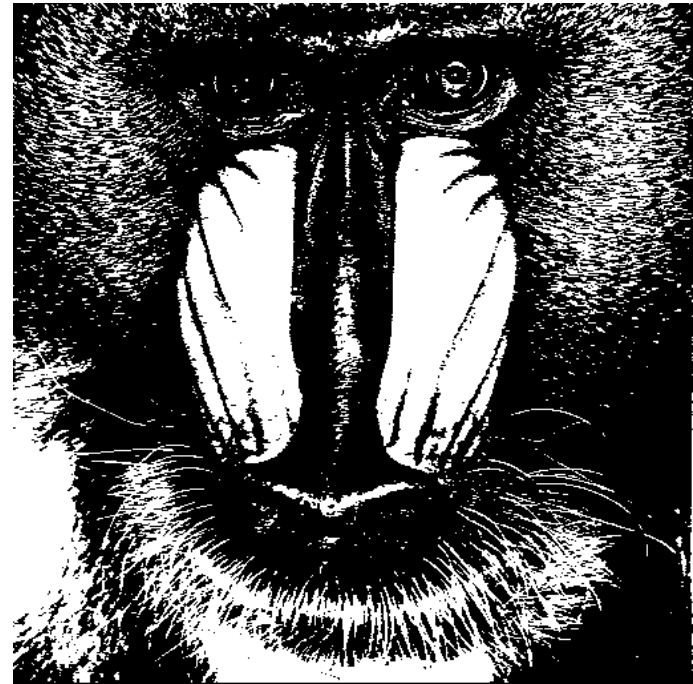
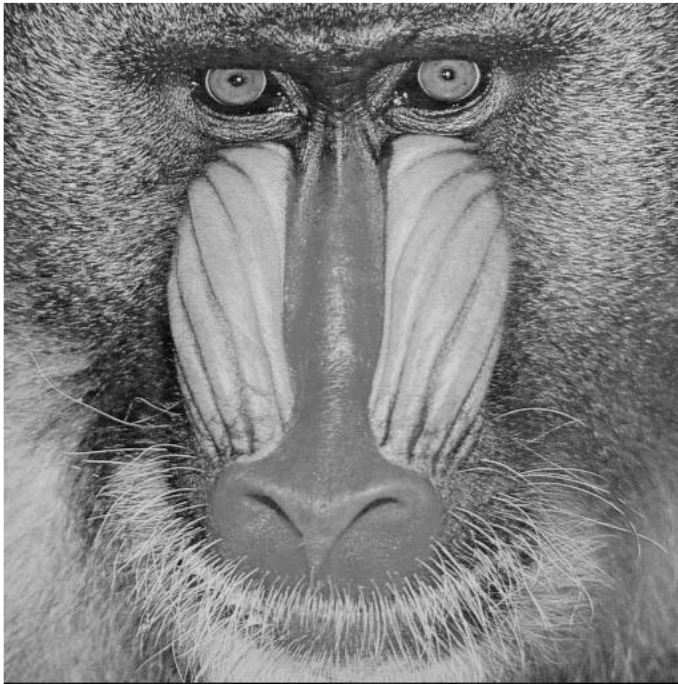
Senão $\text{cor}(i) = \text{Branco (255)}$



**Valor de Corte = 127
(Threshold, Limiar)**

Binarização

Corte = 150



Binarização

Corte = 80



Corte = 100



Corte = 128

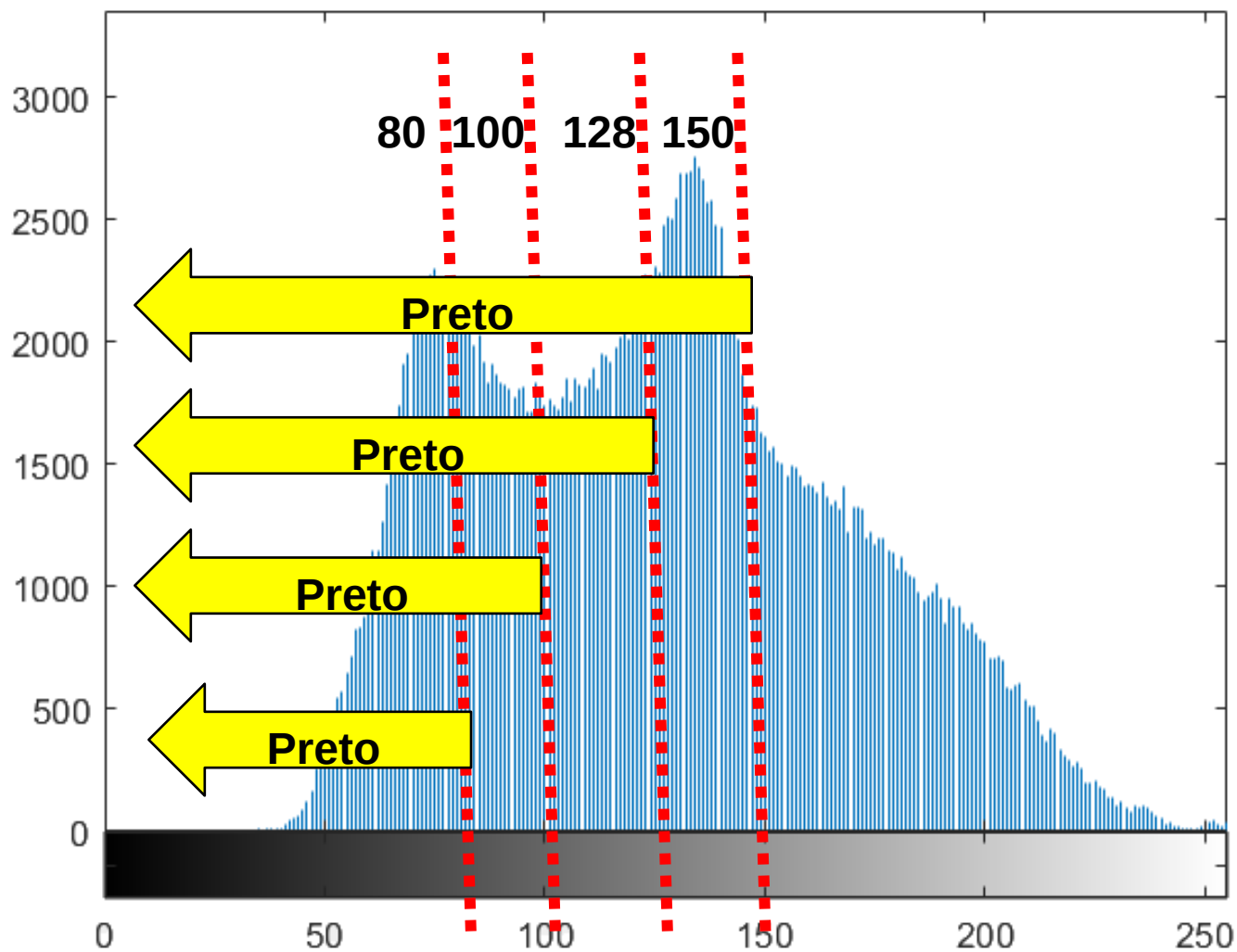


Corte = 150



Corte = limiar de binarização

Binarização

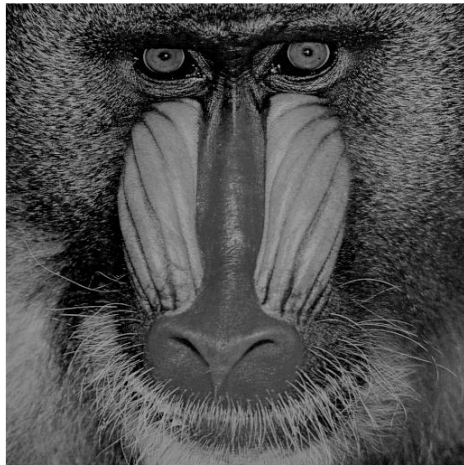


Brilho

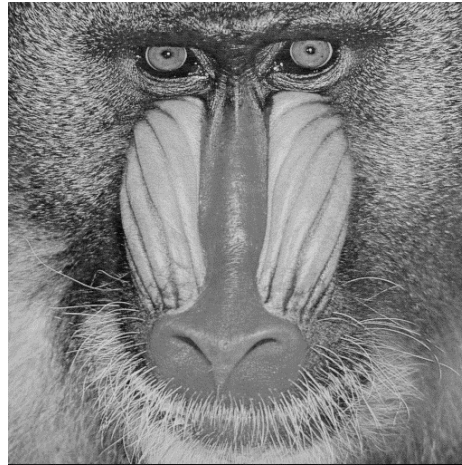
- Mudança de brilho
 - I = Imagem em 256 tons de cinza
 - $I_{out} = I + lum$

Brilho

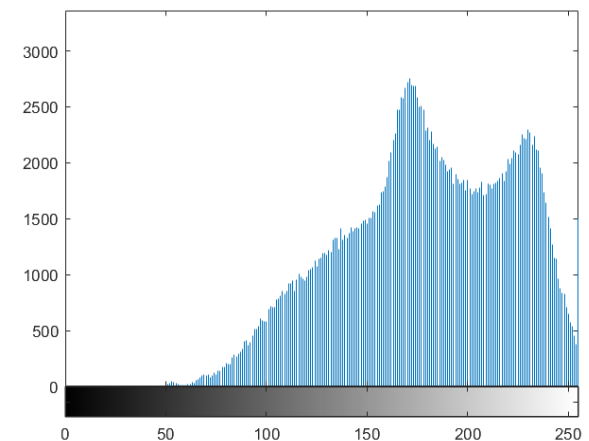
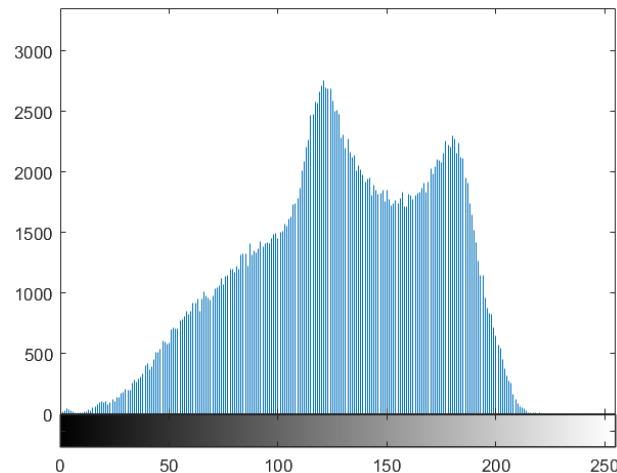
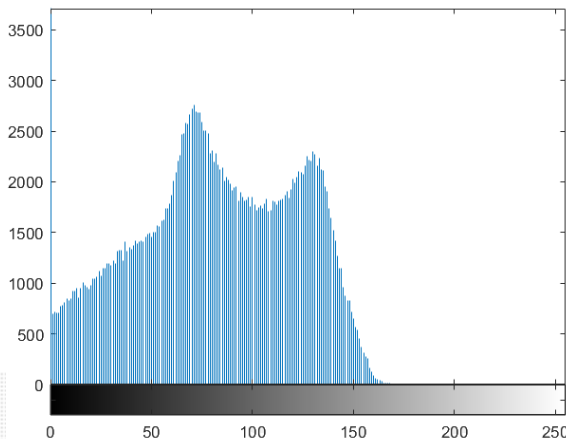
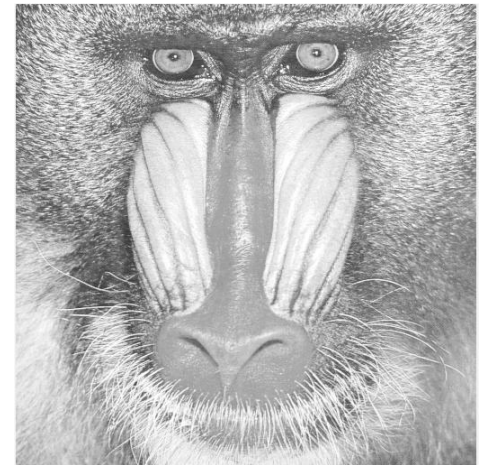
Lum = -50



Lum = 0



Lum = +50

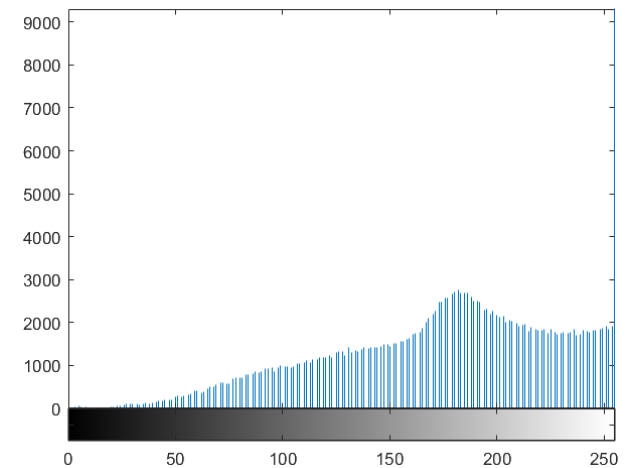
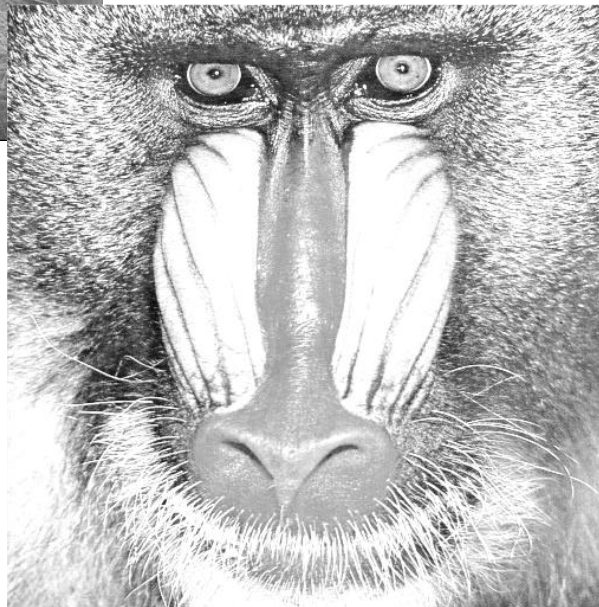
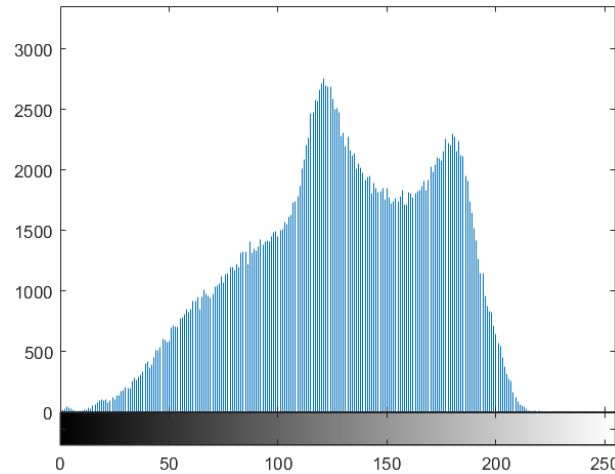
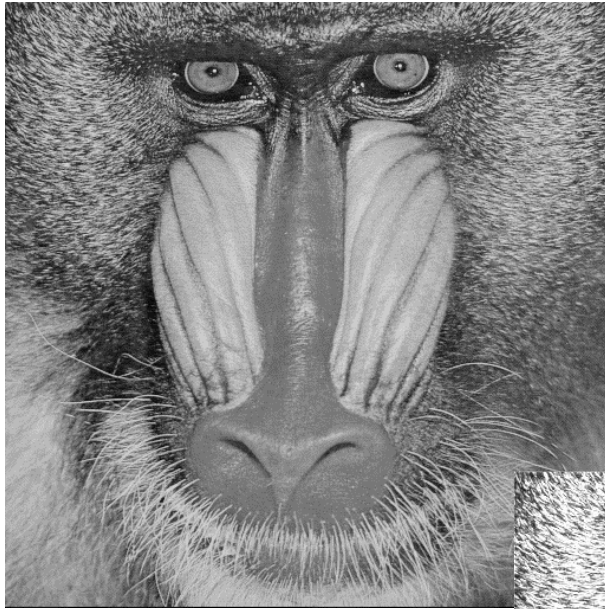


Contraste

- Mudança de Contraste

- I = Imagem em 256 tons de cinza
- $I_{\text{out}} = \text{contr} \times I$
 - Expansão de histograma: $\text{contr} > 1$
 - Compressão de histograma: $0 < \text{contr} < 1$

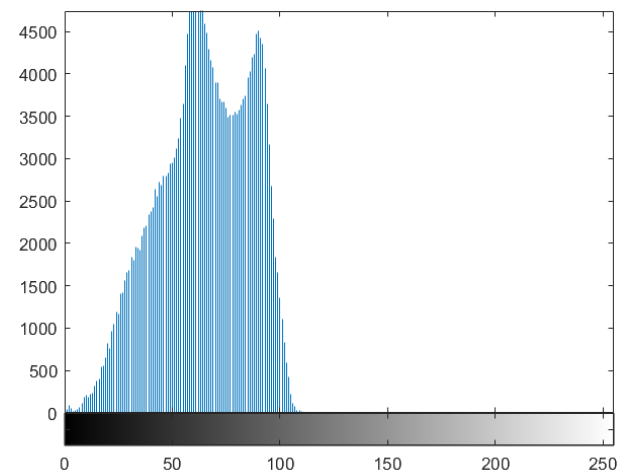
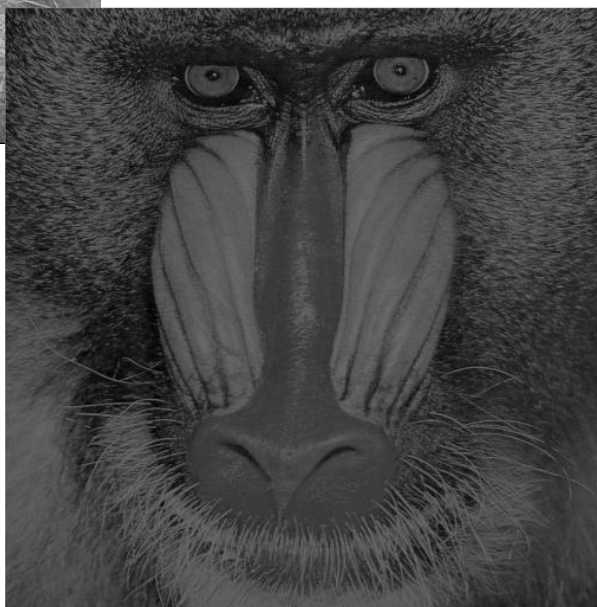
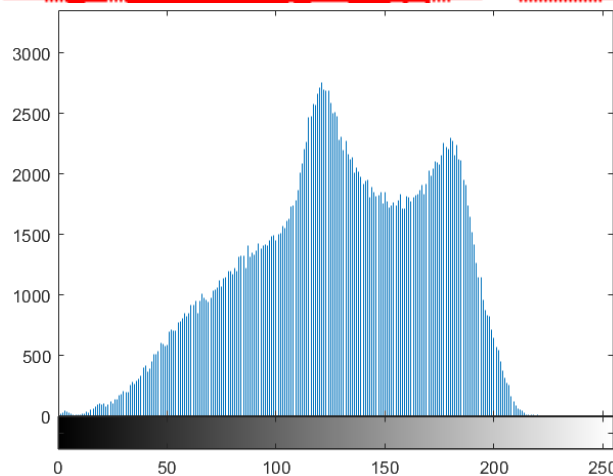
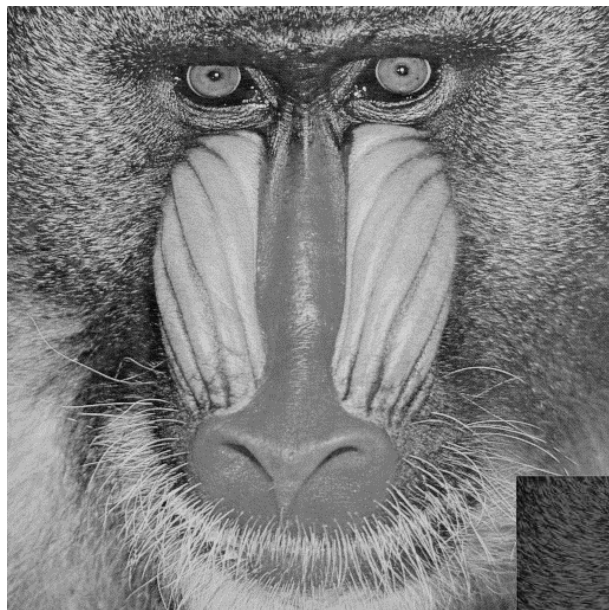
Expansão de histograma



contr = 1,5

Aumenta o contraste

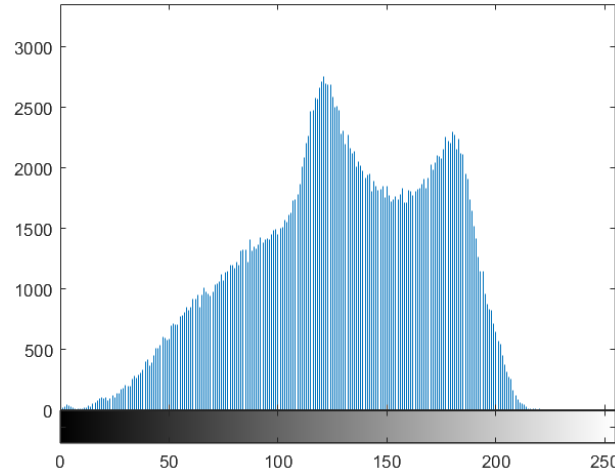
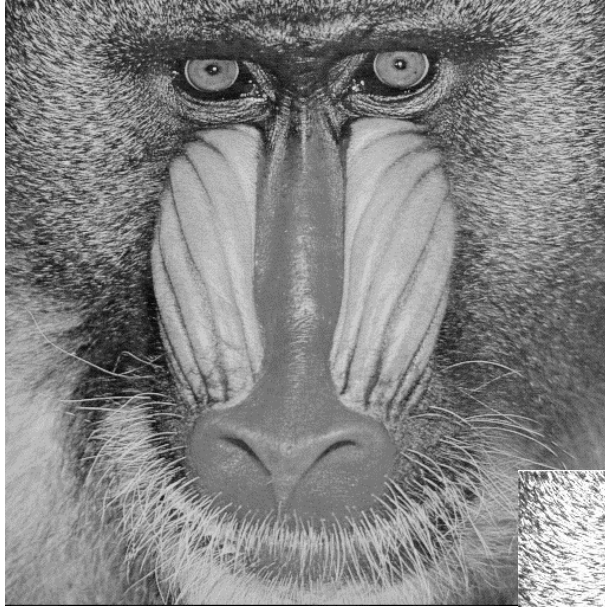
Compressão de histograma



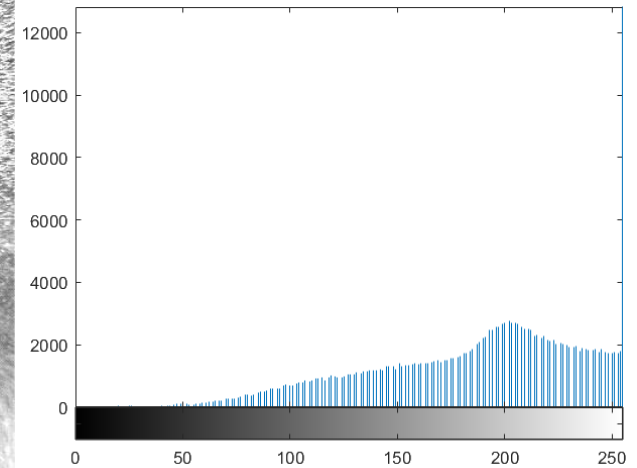
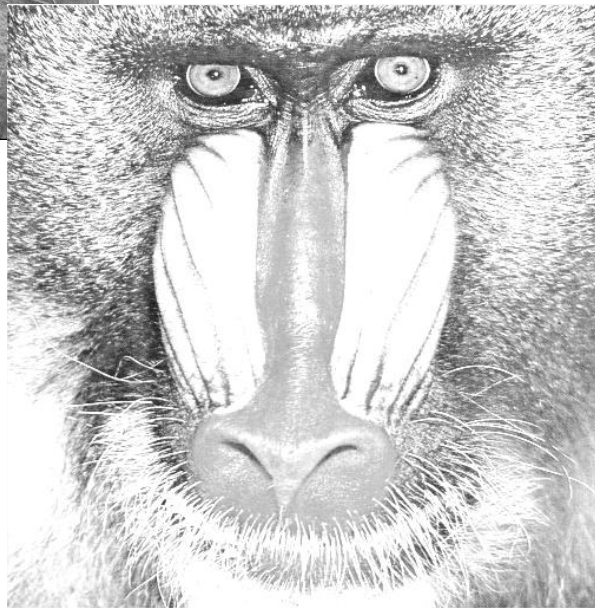
Contr = 0,5

Diminui o contraste

Contraste + Brilho

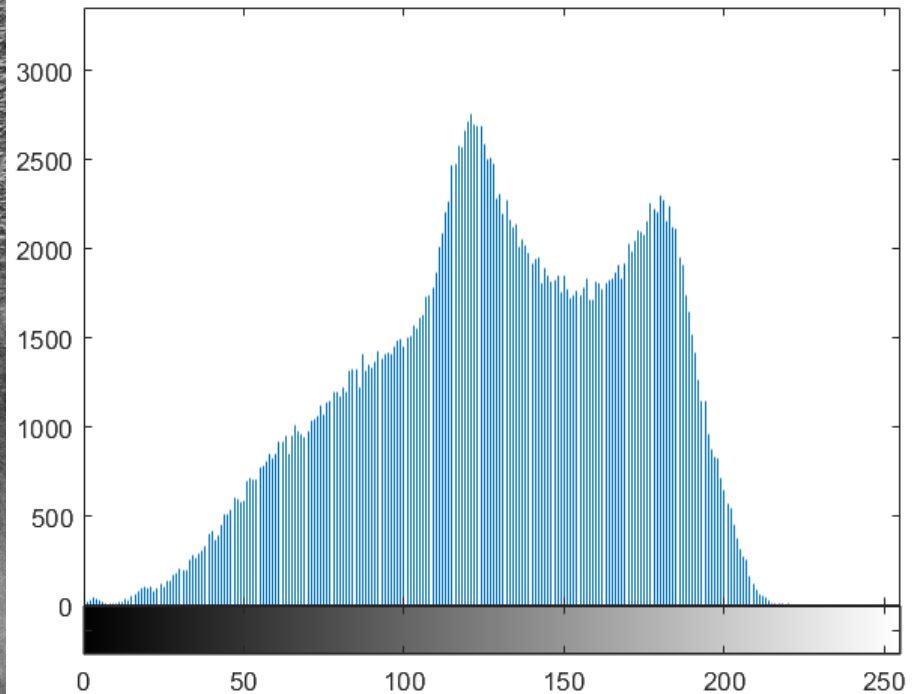
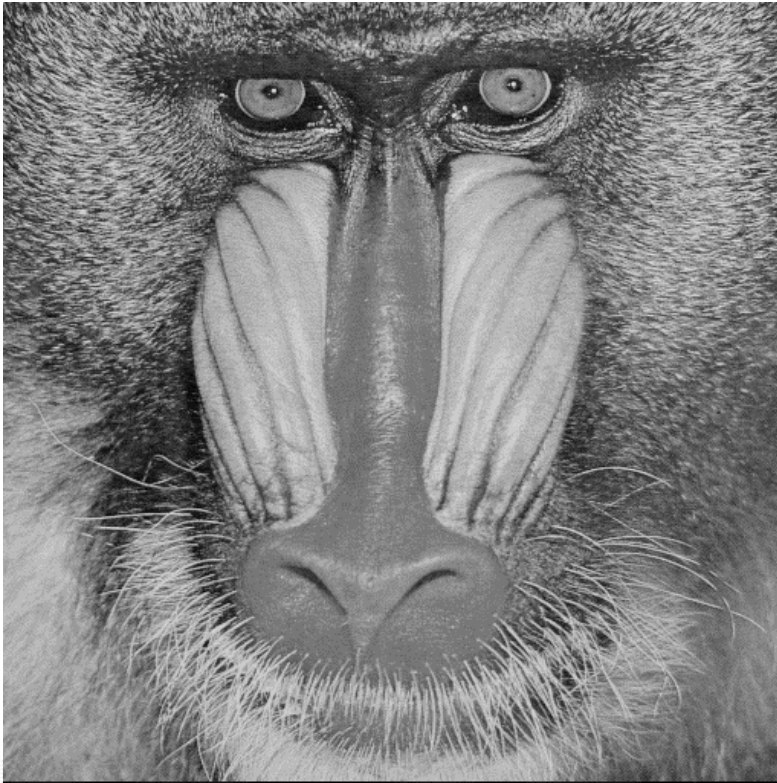


Contr = 1,2
Brilho = +20



Dois Picos do Histograma

- Operação comum em histograma
 - Encontrar os dois maiores picos do histograma



Dois Picos do Histograma

- Dois maiores picos

- Primeiro pico:

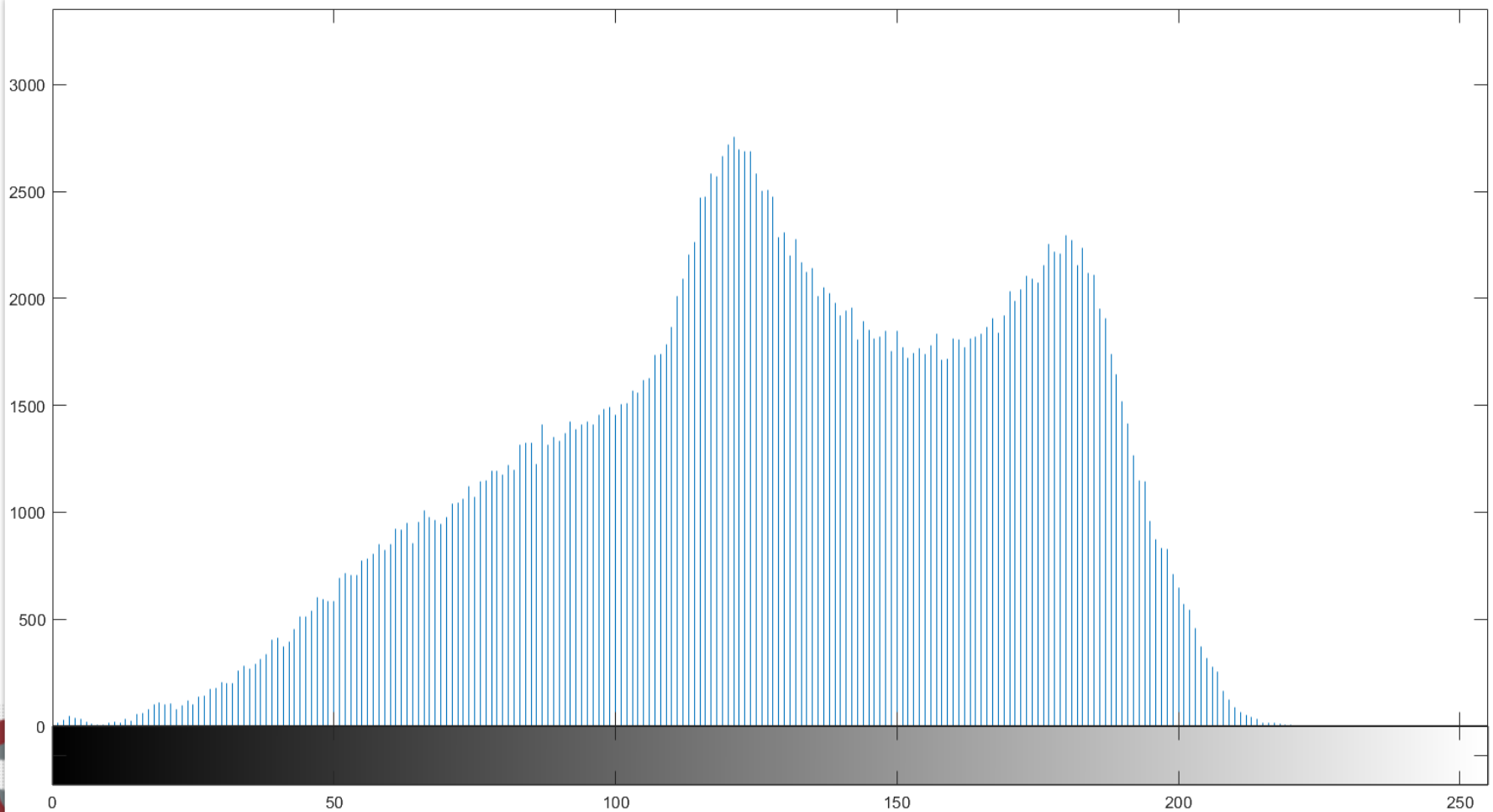
- Maior amplitude do histograma (posição j)

- Segundo pico:

- Não pode ser a segunda maior amplitude
 - Geralmente, próxima da maior amplitude
 - Solução:
 - Segundo pico em: $\max((k - j)^2 h[k])$
 - h é o histograma
 - $0 \leq k \leq 255$ (para imagens com 256 tons de cinza)

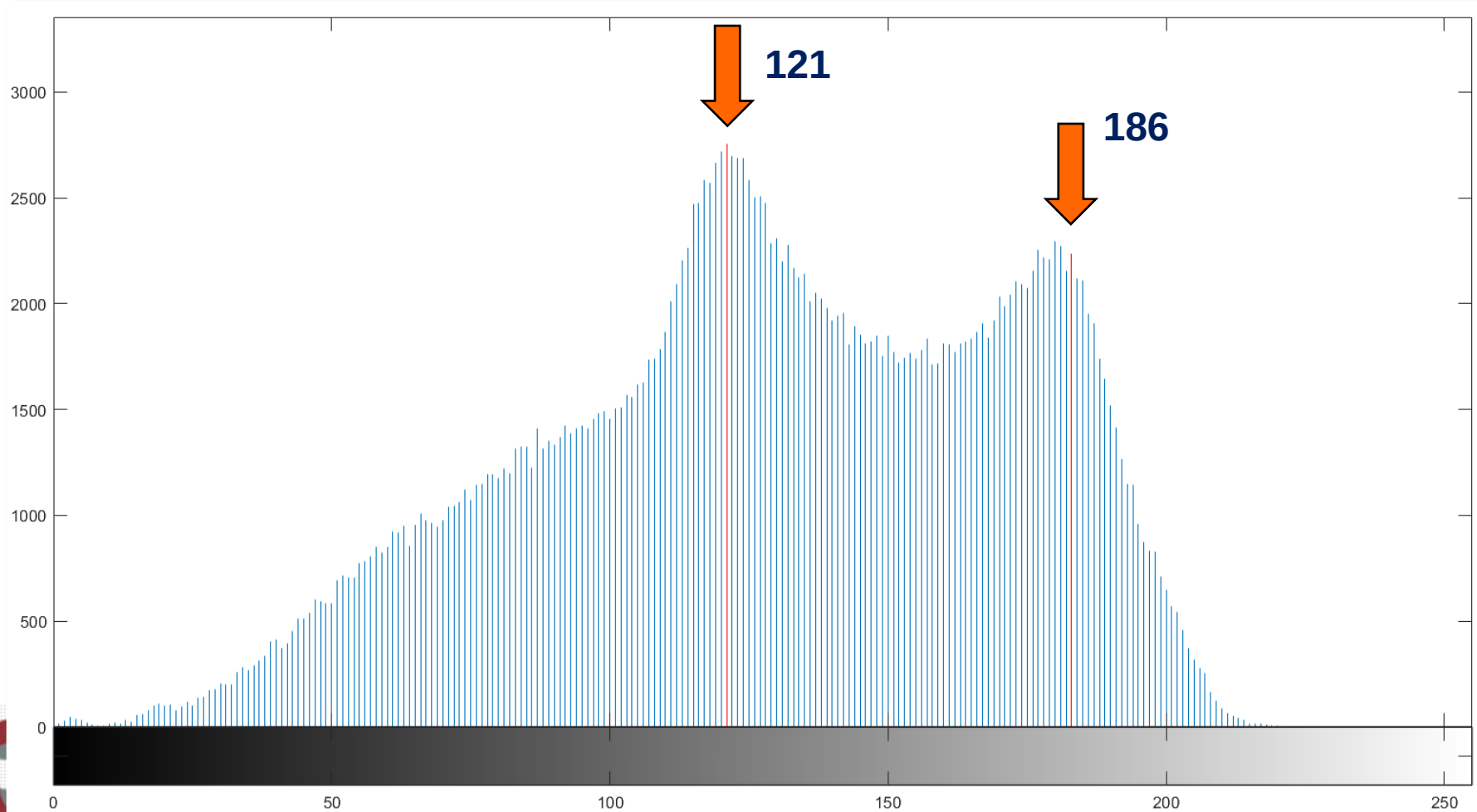
Dois Picos do Histograma

- Dois maiores picos



Dois Picos do Histograma

- Dois maiores picos



Extração de Características

- A partir do histograma:
 - Média
 - Variância
 - Desvio Padrão
 - Entropia
 - Máximo/Mínimo
 - Maiores picos

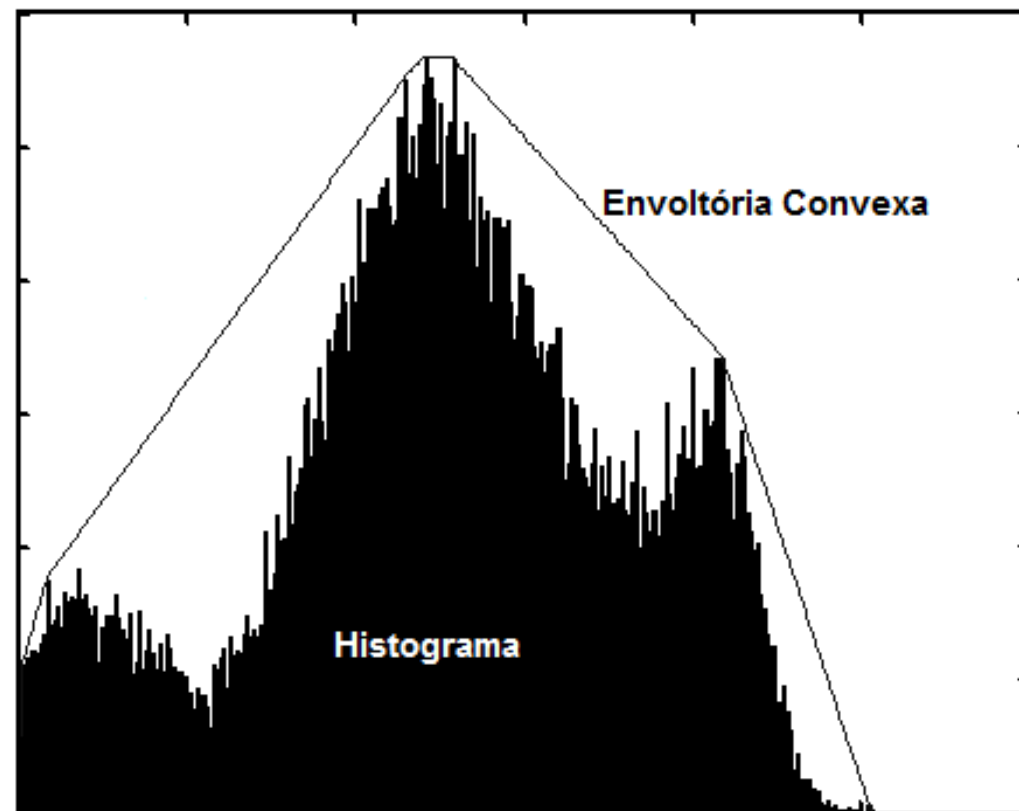
Extração de Características

- Cálculo da Média

- É a média do histograma, não a média dos valores (cores) que aparecem na imagem
- Seja N o número de pixels da imagem
- Seja h o histograma previamente calculado
- Seja $Prob$ a probabilidade de cada cor estar presente na imagem, ou seja, $Prob = h[i]/N$
- Média = $\sum Prob(i).Cor(i)$

Envoltória Convexa (*Convex Hull*)

- Encontrar a forma da envoltória de um histograma



Envoltória Convexa (*Convex Hull*)

- Encontrar a forma da envoltória de um histograma
 - Menor conjunto convexo de pontos que envolve o histograma
 - Problema da área de geometria computacional
 - Análise do “determinante”:
 - Se o determinante é positivo, temos uma guinada no sentido anti-horário
 - Se é negativo, a guinada é no sentido horário
 - Se é nulo, os pontos são colineares
 - Neste caso, o algoritmo pode eliminar o ponto intermediário, que é redundante.

Envoltória Convexa (*Convex Hull*)

- Determinante:

- Início:

- $\text{pos}_1 = 1; \text{pos}_2 = 2; \text{pos}_3 = 3$

- Variações:

- $\text{hist}(\text{pos}_2) - \text{hist}(\text{pos}_3)$

- $\text{hist}(\text{pos}_3) - \text{hist}(\text{pos}_1)$

- $\text{hist}(\text{pos}_1) - \text{hist}(\text{pos}_2)$

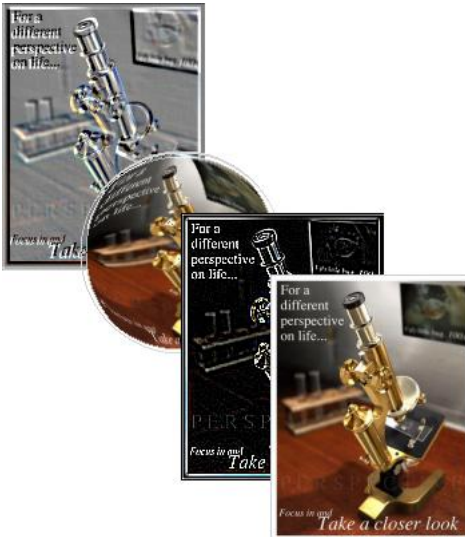
- Determinante:

- $\text{det} = \text{pos}_1 * (\text{hist}(\text{pos}_2) - \text{hist}(\text{pos}_3)) + \text{pos}_2 * (\text{hist}(\text{pos}_3) - \text{hist}(\text{pos}_1)) + \text{pos}_3 * (\text{hist}(\text{pos}_1) - \text{hist}(\text{pos}_2))$

- $\text{pos}_1 \leftarrow \text{pos}_1 + 1$

Limiarização - *Thresholding*

Carlos Alexandre Barros de Mello
CIn/UFPE



Limiarização - *Thresholding*

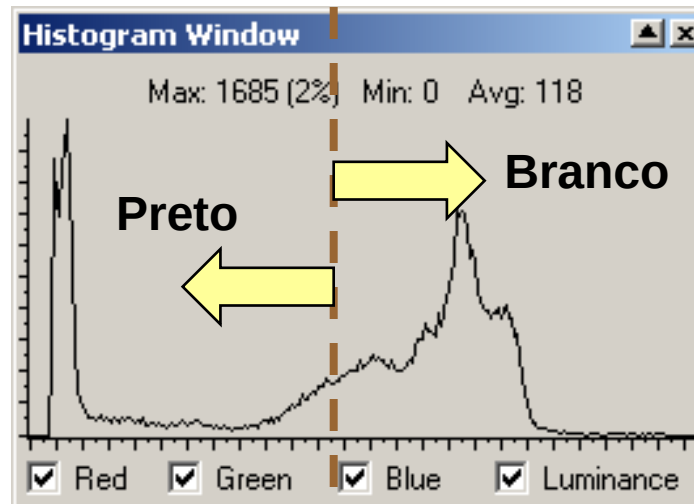
- Conversão de uma imagem para dois tons a partir de um dado ponto de corte (limiar)



Limiarização

- Algoritmo de Recorte

**Se $\text{cor}(i) \leq 127$
Então $\text{cor}(i) = \text{Preto}$
Senão $\text{cor}(i) = \text{Branco}$**

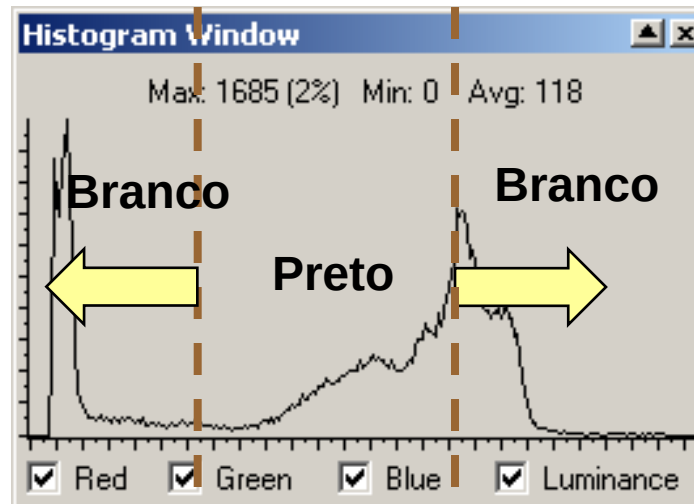


**Valor de Corte = 127
(threshold, limiar)**

Limiarização

- Algoritmo de Recorte (faixa de corte)

**Se $50 < \text{cor}(i) < 100$
Então $\text{cor}(i) = \text{Branco}$
Senão $\text{cor}(i) = \text{Preto}$**



Branco entre 50 e 100

Limiarização

- Tipos de Limiarização

- **Global**

- Um único valor de corte é definido para toda a imagem

- Vantagem: Velocidade
 - Desvantagem: Qualidade

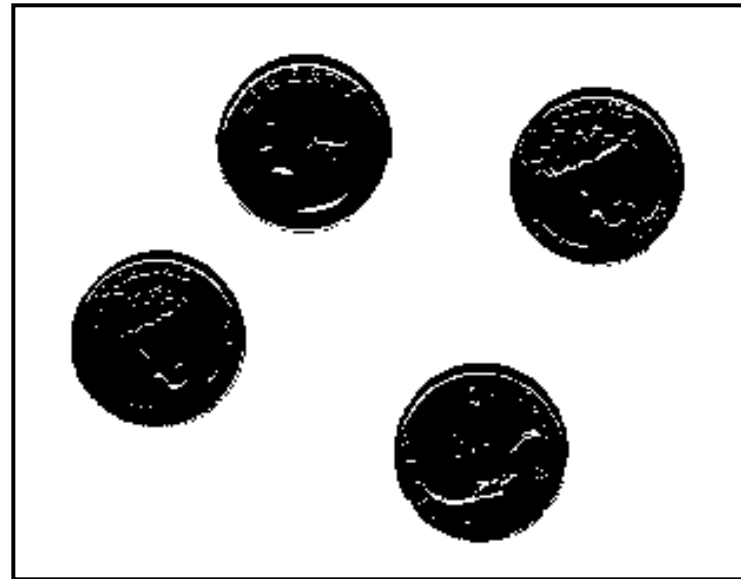
- **Local**

- Diferentes pontos de corte são definidos para diferentes regiões da imagem

- Vantagem: Qualidade
 - Desvantagem: Velocidade

Limiarização

- Limiarização Ótima



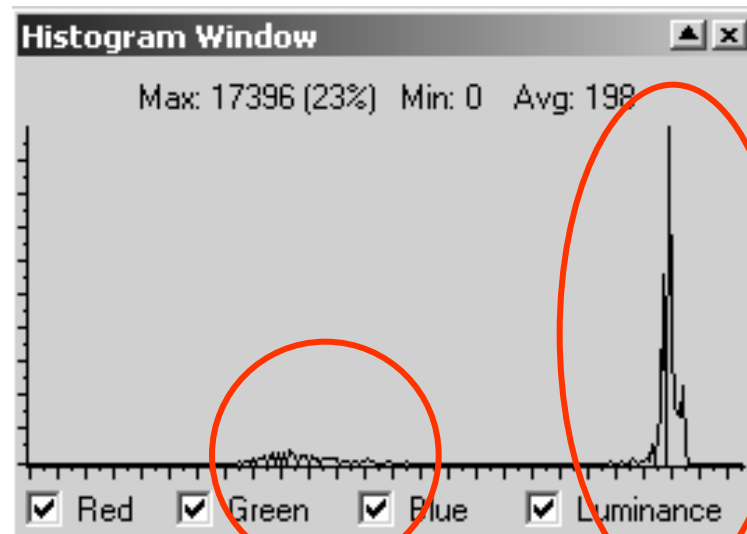
**Objetos formados por tons bem diferentes do background:
Fácil redução de cores com qualidade**

Limiarização

- Limiarização Ótima



Imagem original com cores bem distintas no Histograma



Região com os tons da moeda

Região com os tons do background

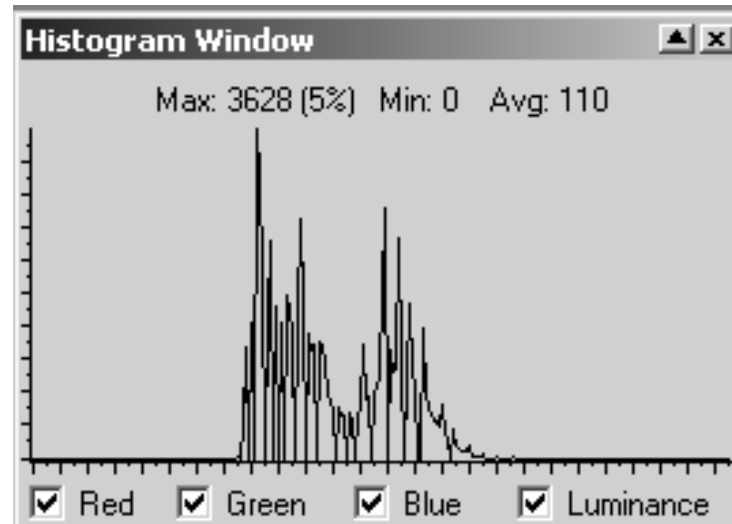
Regiões bem separadas

Limiarização

- Problemas na Limiarização



Problema: Imagem com baixo contraste



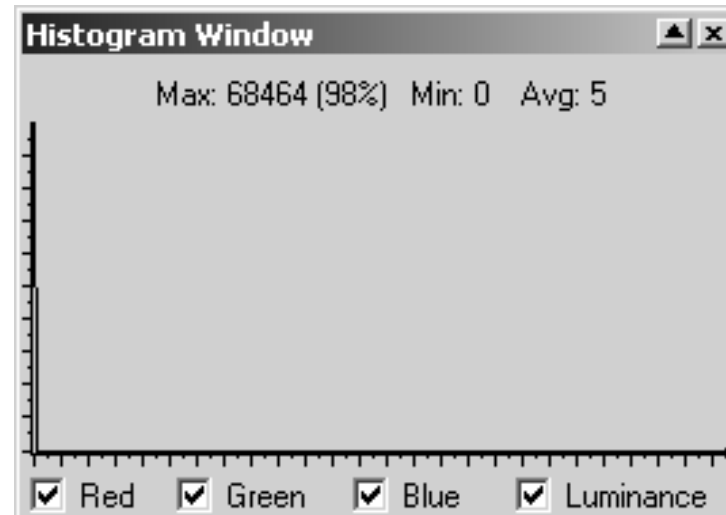
É possível, por exemplo, separar automaticamente a menina do background?

Limiarização

- Problemas na Limiarização

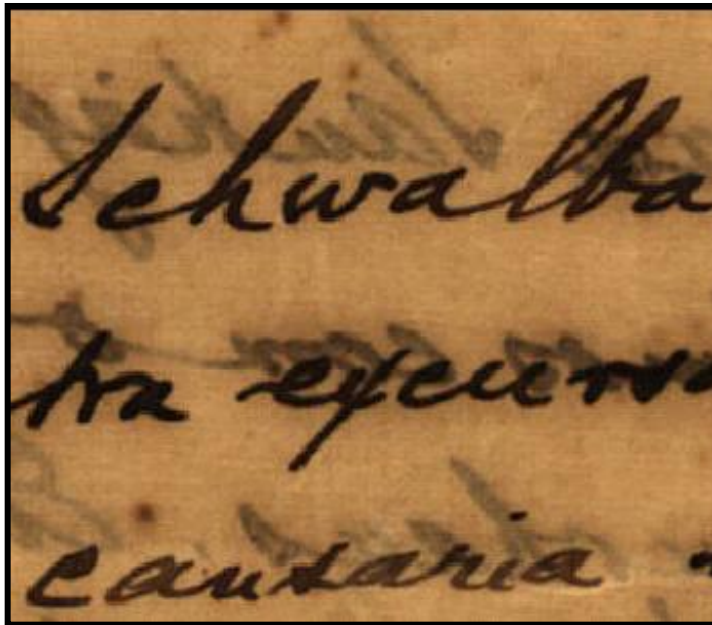


Resultado da Limiarização



Limiarização

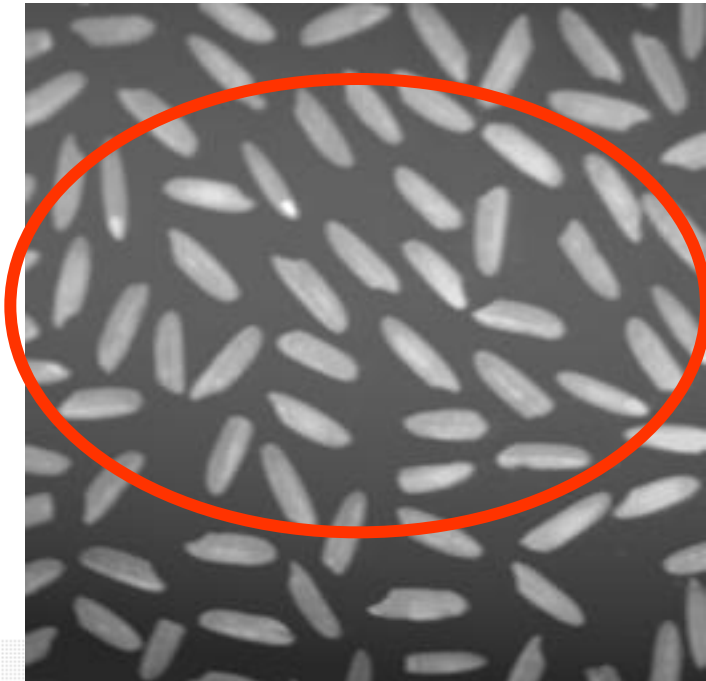
- Problemas na Limiarização



Algoritmo de threshold com
valor de corte padrão

Limiarização Local

- O valor de corte varia dentro de uma mesma imagem, dependendo de fatores da imagem
 - Exemplo: Imagem com diferentes focos de iluminação



Área iluminada

Limiarização

- Os algoritmos também se dividem de acordo com as características procuradas na imagem para gerar o ponto de corte:
 - Histograma
 - Entropia
 - Agrupamento
 - Maximização/Minimização de funções
 - Atributos de objetos
 - Métodos espaciais

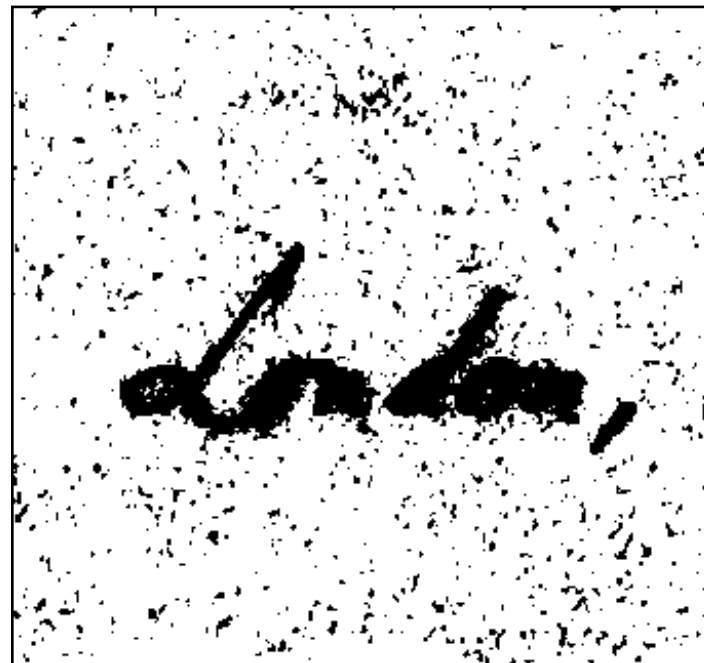
Tom de Cinza Médio

- *Mean Grey Level*
- O ponto de corte é, simplesmente, o ponto médio entre a maior e a menor cor presentes no histograma
 - Não é a média do histograma como calculado antes

Porcentagem de Preto

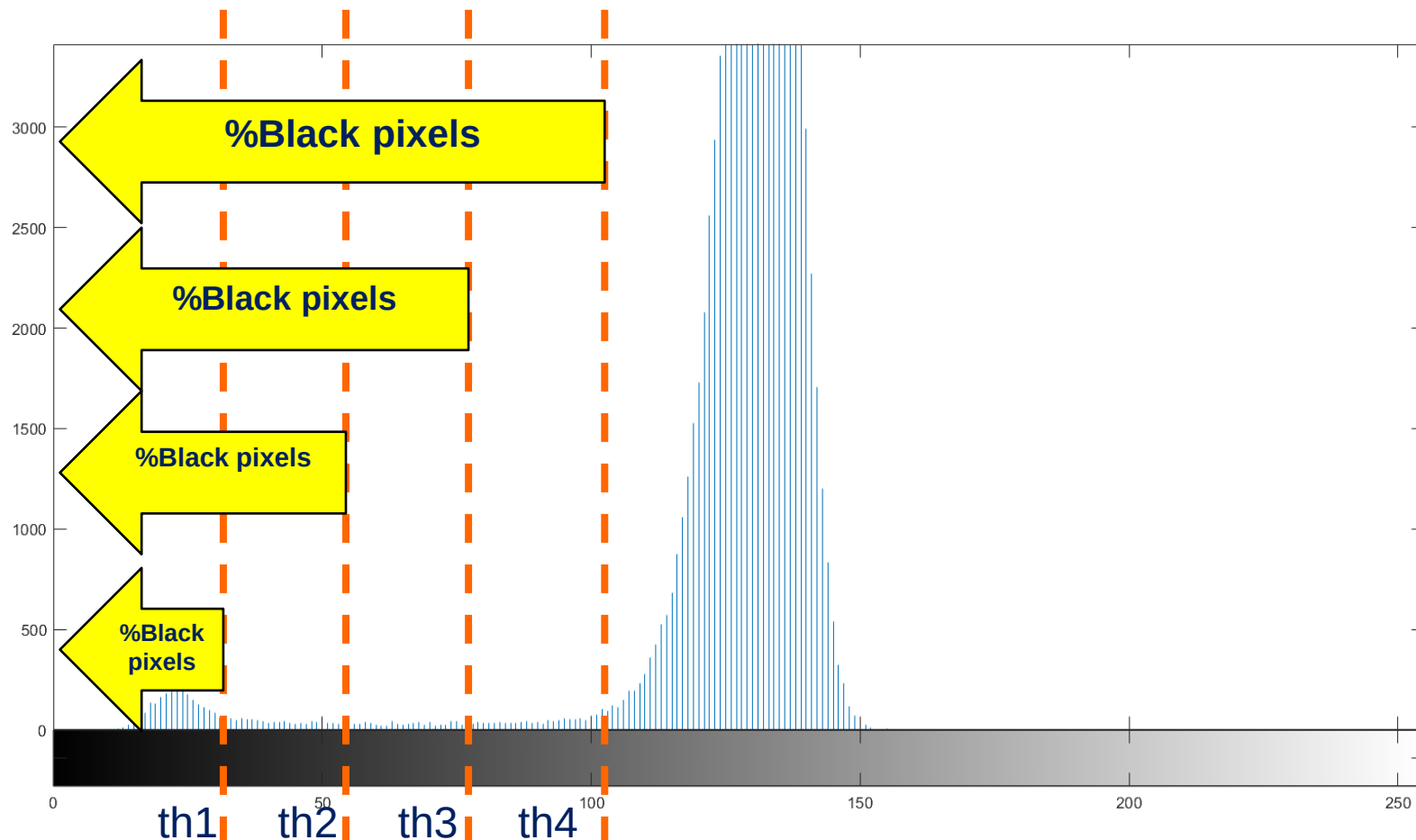
- Define-se *a priori* a quantidade de tons pretos esperados na imagem final
- O ponto de corte é variado até que essa porcentagem seja encontrada
- Exemplo: Valor esperado para imagens de **documentos** é de 10% de preto
- Busca e análise feitas no histograma da imagem

Porcentagem de Preto



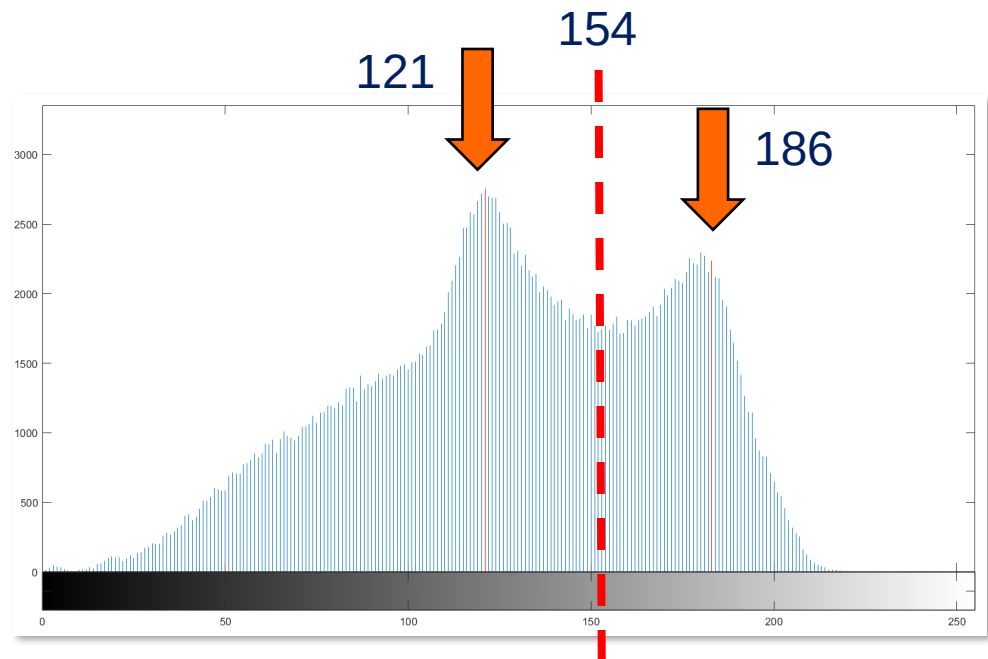
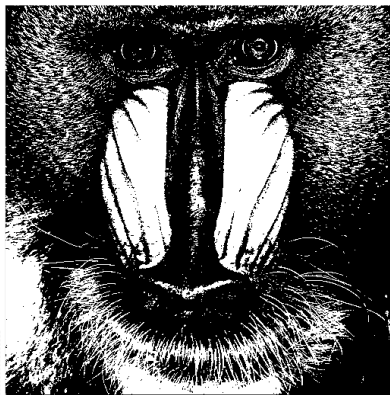
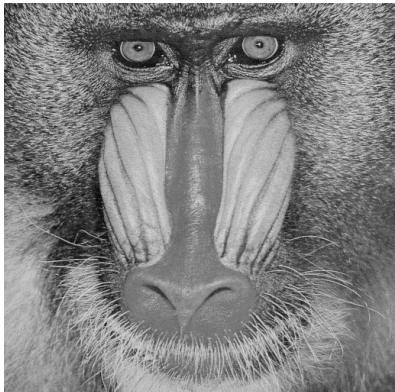
th = 119 (10% de preto)

Porcentagem de Preto



Dois Picos (*Two Peaks*)

- O ponto de corte é o ponto mais baixo entre dois picos
- Localização dos picos como apresentado anteriormente
- Exemplo anterior:



Seleção Iterativa (Thrussel)

- Um *threshold* inicial é atribuído e ajustado através de leituras sucessivas da imagem
 - Não é baseado em histograma
 - Considera 2 classes: *foreground* e *background*
 - *Threshold* inicial (T_0): tom de cinza médio
 - Calcula então o tom de cinza médio abaixo e acima desse T_0 : T_b e T_w
 - Um novo *threshold* é estimado como $(T_b + T_w)/2$
 - O processo continua até que nenhuma mudança ocorra no ponto de corte

Otsu

- A operação de limiarização é considerada como sendo o particionamento dos pixels de uma imagem de L níveis de cinza em duas classes, C_0 e C_1 , que podem representar o objeto e o fundo, ou vice-versa, sendo que esta partição se dará no nível de cinza t
 - Desta forma, teremos $C_0 = \{0, 1, \dots, t\}$ e $C_1 = \{t + 1, t + 2, \dots, L\}$

Otsu

- Seja σ^2_W a variância inter-classe, σ^2_B a variância entre classes e σ^2_T a variância total:

$$p(i) = \frac{cor(i)}{N}$$

$$\mu_t(t) = \sum_{i=0}^t i.p(i)$$

$$\mu_T = \sum_{i=t+1}^{255} i.p(i)$$

$$w_0(t) = \sum_{i=0}^t p(i)$$

$$w_1(t) = 1 - w_0(t)$$

$$\mu_b = \frac{\mu_t(t)}{w_0(t)}$$

$$\mu_w = \frac{\mu_T - \mu_t(t)}{1 - w_0(t)}$$

$$\sigma_b^2 = w_0(t)w_1(t)[\mu_b - \mu_w]^2 \quad \sigma_t^2 = \sum_{i=0}^{255} (i - \mu_t)^2 p(i)$$

$$\sigma_w^2(t) = \sigma_t^2 - \sigma_b^2$$

t é um parâmetro
que vai maximizar
uma função...

Otsu

- O valor ótimo do limiar pode ser encontrado pela maximização da função $\eta(t) = \sigma_b^2(t) / \sigma_T^2$
- Ou seja, a relação entre a variância entre classes e a variância total
- O valor máximo de η , pode ser usado como medida para avaliar a separabilidade das classes C_0 e C_1 na imagem original ou a bimodalidade do histograma

Otsu

- Esta é uma medida bastante significativa pois é invariante para transformações afins da escala de níveis de cinza, sendo unicamente determinada dentro do intervalo $0 \leq \eta \leq 1$
 - O limite inferior (zero) é obtido quando e somente quando uma dada imagem tenha um único e constante nível de cinza, e o limite superior (um) é obtido quando e somente quando imagens de dois valores são dadas (ou seja, já binarizadas)

Otsu

- O método se caracteriza por sua natureza não paramétrica e não supervisionada de seleção de limiar e tem as seguintes vantagens:
 - O processo como um todo é muito simples;
 - Uma extensão direta para problemas que exijam vários limiares é viabilizada dado critério no qual o método está baseado (análise de discriminantes);
 - Um limiar ótimo é selecionado de forma automática e estável, baseado em propriedades globais do histograma;
 - **O método é extremamente genérico**, podendo ser aplicado em qualquer problema relacionado a classificação dentre duas classes

Usando Pixels de Borda

- Baseado no operador Laplaciano (Filtro Passa Alta)

0	1	0
1	-4	1
0	1	0

- Após a filtragem, a imagem é binarizada
- O histograma da imagem filtrada é calculado, considerando apenas aqueles pixels com “alto” valor de laplaciano
 - “alto” é um percentual pré-definido
 - A binarização é feita usando esse histograma

Usando Pixels de Borda

- Funciona como uma mistura de dois outros algoritmos
 - Como no algoritmo de porcentagem de preto, procura o ponto de corte que passaria para preto o percentual pré-definido da imagem
 - Mas, no caso, esse ponto definirá apenas a parte do histograma que será processada
 - Nesse pedaço de histograma, usa o algoritmo de dois picos para achar o ponto de corte

Média Móvel (*Moving Average*)

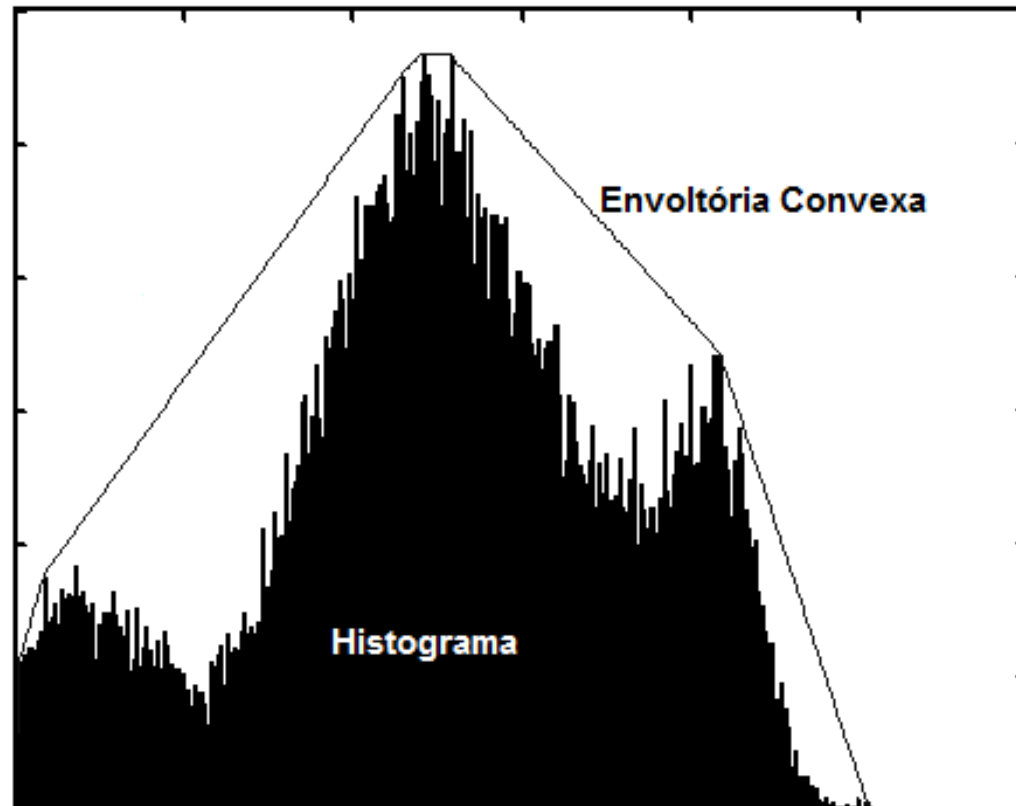
P.Wellner

- Limiarização Local
- Define um ponto de corte para cada pixel
- Uma média móvel é apenas o tom de cinza médio dos últimos n pixels
- Nesse caso, a imagem pode ser considerada armazenada como um vetor
- Para o pixel $i+1$ com cor $cinza_{i+1}$:
 - $M_{i+1} = M_i - M_i/n + cinza_{i+1}$

Rosenfeld

- Baseado na análise da envoltória convexa do histograma de uma imagem
 - A envoltória convexa de um histograma é o menor polígono convexo para o qual todos os níveis do histograma estejam no seu interior
- No algoritmo de Rosenfeld, a maior diferença entre a envoltória e o histograma se torna candidato ao limiar
 - Se houver mais de um candidato, outras características da imagem podem ser definidas como padrões de desempate

Rosenfeld

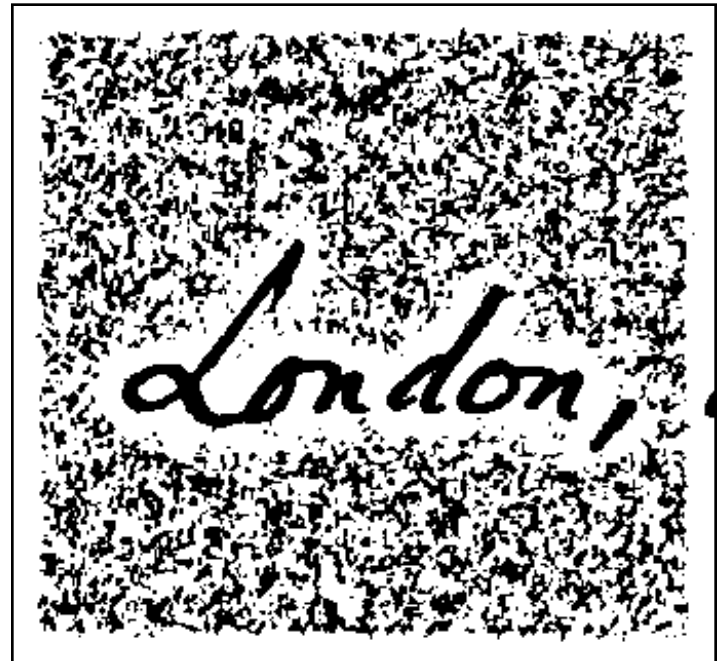


Niblack

- Limiarização Local
- Imagem dividida em janelas quadradas ($N \times N$) e cada janela é avaliada em separado
- Calcula-se média (m) e desvio padrão (d) de cada janela
- O ponto de corte (th) é definido por:
 - $th = m + bias \cdot d$
 - $bias$ = peso atribuído
 - Não há definição de valor específico para esse *bias*

Niblack

- Exemplo:



Janela = 31; Bias = -0.8

Sauvola

- Melhoria no algoritmo de Niblack para imagens com problemas de iluminação
- Nova variável para modificar o efeito do desvio padrão
 - $R = 128$
- O ponto de corte é definido como:
 - $th = m + bias \cdot (d/R)$

White

- Compara o tom de cinza de cada *pixel* da imagem com a média dos valores dos seus vizinhos numa janela
- Para peso igual a 2, se o *pixel* for mais escuro que a média, ele é classificado como objeto, senão é background
 - Se $\text{média} > \text{cor_pixel} \times \text{peso}$, então $\text{cor_pixel} \leftarrow \text{Preto}$
 - Senão $\text{cor_pixel} \leftarrow \text{Branco}$

Bernsen

- O limiar é escolhido como a média entre o mínimo e o máximo tom de cinza da vizinhança do *pixel*
- Se o contraste for menor que certo limiar dado, então o *pixel* é considerado da mesma classe que seus vizinhos, objeto ou *background*, dependendo do valor do limiar
 - O contraste é calculado como a diferença entre o máximo e o mínimo nível de cinza dado

Avaliação de Limiarização

- Problema: como avaliar algoritmos?
- Domínio específico das imagens
 - Documentos tipografados: resposta de OCR
- E os outros domínios?
 - Imagens gerais
 - Documentos manuscritos

Avaliação de Limiarização

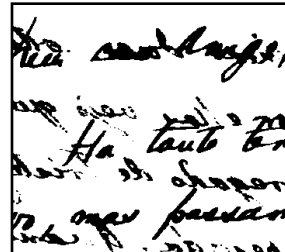
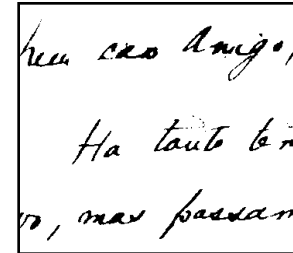
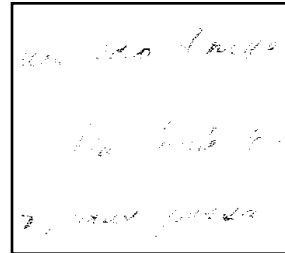
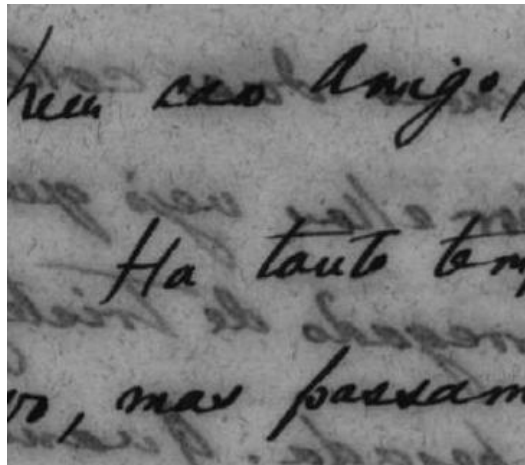
- Uso de *ground truth* ou *gold standard*
 - O padrão ouro
- Problema: como gerar? Quantos gerar?
 - Sem respostas....
 - Mesmo com o *ground truth*, como comparar?
 - Comparação pixel a pixel é eficiente
 - Mas deve-se comparar acertos e erros

Avaliação de Limiarização

- Comparação do valor do ponto de corte
- Avaliação do erro do calculado e do esperado
 - Uso do *ground truth* novamente
 - E o que acontece com algoritmos de binarização local?

Avaliação de Limiarização

- Exemplo: Imagens de documentos manuscritos



- Para diferentes pontos de corte, temos diferentes imagens resultantes

Avaliação de Limiarização

- Uso de medidas de teoria da detecção de sinais
 - Um pixel corretamente considerado como tinta e convertido para preto (*TP – True Positive*)
 - Um pixel corretamente considerado como papel e convertido para branco (*TN – True Negative*)
 - Um pixel de tinta considerado papel e convertido para branco (*FN – False Negative*)
 - Um pixel de papel considerado tinta e convertido para preto (*FP – False Positive*)

Avaliação de Limiarização

- Comparação com o *ground truth*
- Imagem ideal criada manualmente
 - Problema: geração das imagens ideais
 - Vantagem: Facilidade com a comparação com os resultados gerados por outros algoritmos

Imagem original

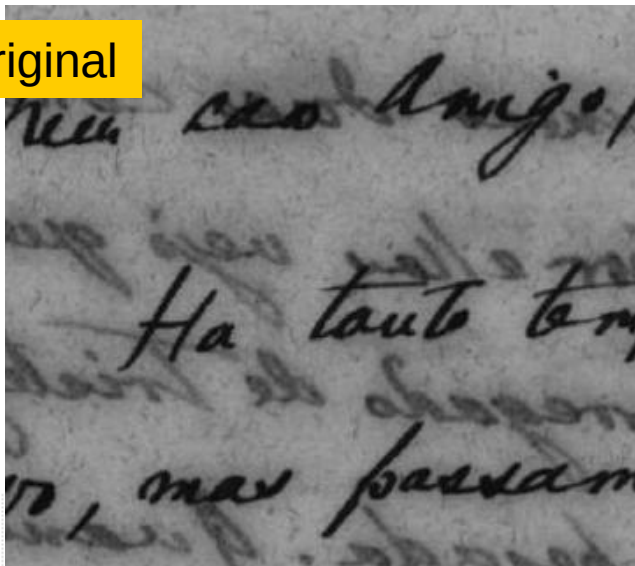
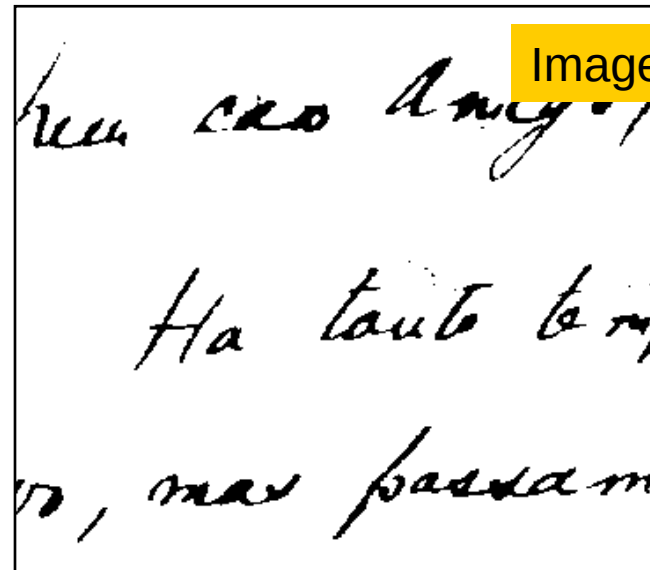
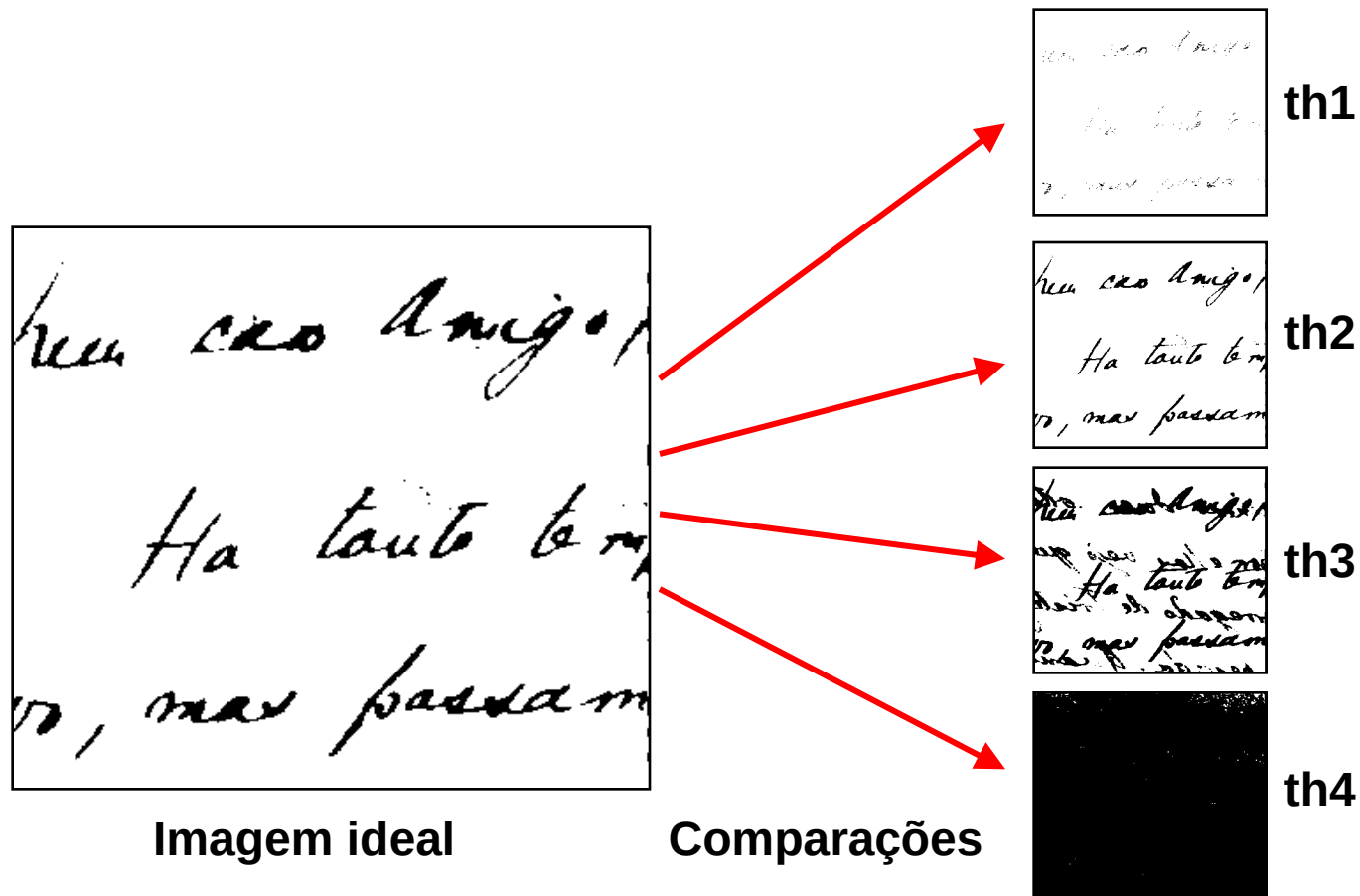


Imagem ideal



Avaliação de Limiarização



Avaliação de Limiarização

- Observe o seguinte caso:

Imagem ideal

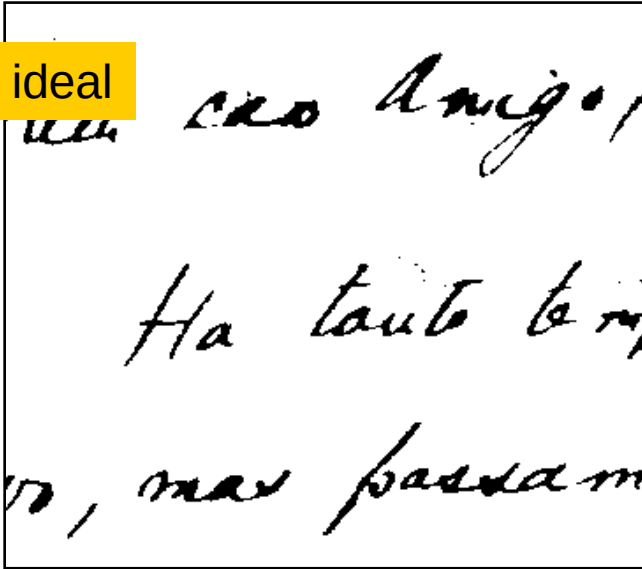


Imagem binarizada



- Exemplo: Nesse caso, todos os pixels pretos da imagem original foram corretamente classificados como tinta (alto TP)
- Mas, houve erro na classificação de todos os pixels brancos (alto FP)
- Ou seja, uma medida apenas não mostra o correto desempenho do algoritmo

Avaliação de Limiarização

- Medidas de desempenho:
 - Precision = $TP / (TP + FP)$
 - Recall = TP / P = Sensitivity
 - Accuracy = $(TP + TN) / (P + N)$
 - Specificity = $TN / (FP + TN)$
 - Onde:
 - $P = TP + FN$
 - $N = FP + TN$
- Para uma boa qualidade de limiarização, todas essas medidas devem se aproximar de 1 (indicando poucos erros)

Avaliação de Limiarização

- Um bom algoritmo deve ter
 - **Precision ≈ 1 :**
 - $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
 - Assim, para *precision* tender a 1, é preciso que FP tenda a zero (ou seja, poucos erros)
 - **Recall ≈ 1 :**
 - $\text{Recall} = \text{TP} / P = \text{TP} / (\text{TP} + \text{FN})$
 - Assim, para *recall* tender a 1, é preciso que FN tenda a zero
 - **Accuracy ≈ 1 :**
 - $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FN} + \text{FP} + \text{TN})$
 - Ou seja, FN e FP devem ser próximos de zero
 - **Specificity ≈ 1 :**
 - $\text{Specificity} = \text{TN} / (\text{FP} + \text{TN})$
 - Ou seja, FP deve ser próximo de zero

DIBCO

- Document Image Binarization Contest
 - DIBCO
 - Imagens de documentos tipografados
 - H-DIBCO
 - H = *Handwritten*
 - Ocorrem anualmente no ICDAR ou no ICFHR
 - Disponibilizam bases de imagens de anos anteriores

DIBCO

- Medidas definidas
 - F-measure:
 - $2 * \text{Recall} * \text{Precision} / (\text{Recall} + \text{Precision})$
 - PSNR:
 - $10 * \log(C^2 / \text{MSE})$
 - MSE = Mean Square Error
 - Negative Rate Metric (NRM):
 - $(\text{NR}_{\text{FN}} + \text{NR}_{\text{FP}}) / 2$
 - $\text{NR}_{\text{FN}} = \text{FN} / (\text{FN} + \text{TP})$
 - $\text{NR}_{\text{FP}} = \text{FP} / (\text{FP} + \text{TN})$

DIBCO

- Medidas definidas

- Misclassification Penalty Metrics (MPM)

- Avalia a predição contra o *ground truth* numa base objeto-a-objeto
- Pixels classificados erroneamente são penalizados pela sua distância da borda do objeto na imagem *ground truth*

$$MPM = \frac{MP_{FN} + MP_{FP}}{2}$$

$$MP_{FN} = \frac{\sum_{i=1}^{N_{FN}} d_{FN}^i}{D}, \quad MP_{FP} = \frac{\sum_{j=1}^{N_{FP}} d_{FP}^j}{D}$$

d_{FN}^i e d_{FP}^j denotam a distância do i^{th} falso negativo pixel e do j^{th} falso positivo pixel do contorno da segmentação GT. O fator de normalização D é a soma de todas as distâncias pixel para contorno do objeto GT. Um baixo MPM indica que o algoritmo é bom na identificação dos contornos do objeto.

Sugestões de Leitura

- A Novel Contrast Enhancement Technique using Gradient-Based Joint Histogram Equalization
 - Circuits, Systems, and Signal Processing, 2021
 - <https://doi.org/10.1007/s00034-021-01655-3>
- A novel equilibrium optimization algorithm for multi-thresholding image segmentation problems
 - Special Issue on Hybridization between Neural Computing and Nature Inspired Algorithms for Solving Multi-Criteria Decision-Making Problems, 2021
 - <https://doi.org/10.1007/s00521-020-04820-y>
- A novel Black Widow Optimization algorithm for multilevel thresholding image segmentation
 - Expert Systems with Applications, 2021
 - <https://doi.org/10.1016/j.eswa.2020.114159>
- An improved opposition-based marine predators algorithm for global optimization and multilevel thresholding image segmentation
 - Knowledge-Based Systems, 2021
 - <https://doi.org/10.1016/j.knosys.2021.107348>

Referência

- Imprescindível para qualquer trabalho que envolva limiarização:
 - Sezgin, M.; Sankur, B., “Survey over image thresholding techniques and quantitative performance evaluation”. Journal of Electronic Imaging, 2004. Vol.13, p.146-165, 2004.

Próxima Aula - Transformadas

